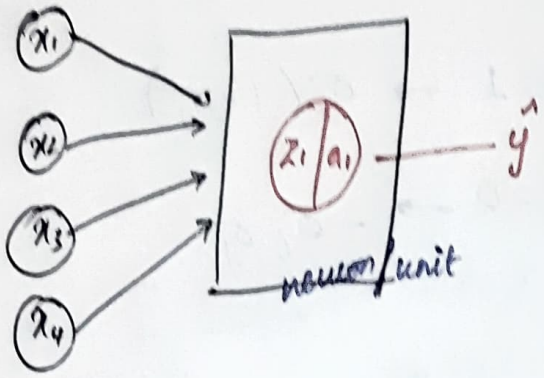
18/04/25

Lab.

I/p vector x

$$\begin{bmatrix} 0.2 \\ -1.3 \\ 2 \\ 0.01 \end{bmatrix}$$

Layer 1



Input (not a neuron)

O/P

Compute $\hat{y}$

linear comb of weights & ip.

$$z_1 = w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + w_{14}x_4$$

$a_1 = \text{Softmax}(z_1)$

↓ non-linear

$$\begin{bmatrix} w_{11} \\ w_{12} \\ w_{13} \\ w_{14} \end{bmatrix} = \begin{bmatrix} 0.001 \\ 0.01 \\ -0.005 \\ -1.2 \end{bmatrix}$$

$$z_1 = 0.001 \times 0.2 + 0.01 \times (-1.3) + 2 \times (-0.005) + 0.01 (-1.2)$$

$$= 0.0002 - 0.013 - 0.01 - 0.012$$

$$= 0.0002 - 0.035$$

$$z_1 = -0.0348$$

$$a_1 = \text{Softmax}(-0.0348).$$

$$= \frac{e^{-0.0348}}{\sum e^{-0.0348}} = 1$$

$$a_1 = 1$$

$$Z = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} \begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} \end{bmatrix}^T$$

$$= \begin{bmatrix} 0.2 \\ -1.3 \\ 2 \\ 0.01 \end{bmatrix} \begin{bmatrix} 0.001 & 0.01 & -0.005 & -1.2 \end{bmatrix}$$

$$z_1 = 0.2 \times 0.001 + (-1.3 \times 0.01) + (2 \times -0.05) + 0.01 \times -1.2$$

$$z_1 = -0.0348$$

For ReLU

$$a_1 = ReLU(z_1)$$

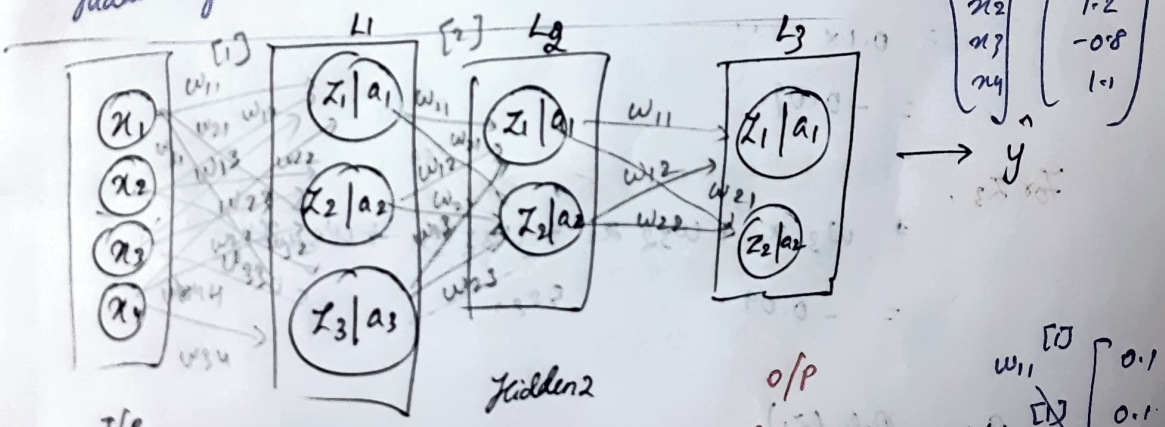
$$a_1 = ReLU(-0.0348)$$

$$a_1 = 0.$$

Feed forward network

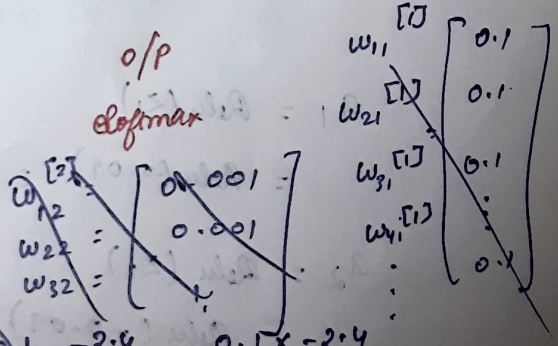
Hidden layers ReLU, output layer softmax

$$\begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} -2.4 \\ 1.2 \\ -0.8 \\ 1.1 \end{pmatrix}$$



12 pm 6

12+6+4 = 22



Layer 1: $w_{11}x_1 + w_{21}x_1 + w_{31}x_1 \ldots$

For x_1 = $0.1 \times -2.4 + \ldots \times 0.1 + \ldots \times 0.1 + 1.1 \times 0.1$

$= -0.24 + 0.12 - 0.08 + 0.11 \rightarrow 0.23 - 0.32$

$= -0.09$

For x_2 → $w_{12}x_2 + w_{22}x_2 + w_{32}x_2 + w_{34}x_2$

$= 0.001 \times 1.2 + \ldots 1.2 \times 0.1 + 1.2 \times 0.1 + 1.2 \times 0.1$

$= 0.48$

For x_3,

$\Rightarrow (0.1 \times -0.8) \times 4$

$= -0.32$

For x_4,

$\Rightarrow 0.1 \times 1.1$

$= 0.11 \times 4$

$= 0.44$

Layer 1

For z_1,

$$w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + w_{14}x_4$$

$= 0.1 \times -2.4 + 0.1 \times 1.2 + 0.1 \times -0.8 + 0.1 \times 1.1$

$= -0.09$

For z_2,

$$w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + w_{24}x_4$$

$= 0.1 \times$

$= -0.09$

For z_3,

$$w_{31}x_1 + w_{32}x_2 + w_{33}x_3 + w_{34}x_4$$

$= -0.09$

$a_1 = \text{Relu}(z_1)$

$= \text{Relu}(-0.09) = 0$

$a_2 = \text{Relu}(z_2)$

$= \text{Relu}(-0.09) = 0$

$a_3 = \text{Relu}(z_3)$

$= \text{Relu}(-0.09) = 0.$

Layer 2:

0.001 $\quad$ weight $= \begin{bmatrix} 0.001 \\ \vdots \end{bmatrix}$ $\quad$ Now x_1, x_2, x_3 is $\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \begin{matrix} x_1 \\ x_2 \\ x_3 \end{matrix}$

~~0.001 × 0~~

For z_1

$\Rightarrow w_{11}x_1 + w_{12}x_2 + w_{13}x_3$

$= 0.001 \times 0 + 0.001 \times 0 + 0.001 \times 0$

$= 0$

For z_{21}

$\Rightarrow w_{21}x_1 + w_{22}x_2 + w_{23}x_3$

$= 0.001 \times 0 + 0.001 \times 0 + 0.001 \times 0$

$= 0.$

$a_1 = Relu(z_1)$

$a_1 = Relu(0) = 0$

$a_2 = Relu(z_2)$

$a_2 = Relu(0) = 0$

Layer 3:

Now $[x_1\ x_2]$ will be $[a_1\ a_2]$ $\quad w[3] = 0.01$

For z_1 $\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \Rightarrow w_{11}x_1 + w_{12}x_2$

$= 0.01 \times 0 + 0.01 \times 0$

$= 0 + 0 \Rightarrow 0.$

$a_1 = Softmax(0)$

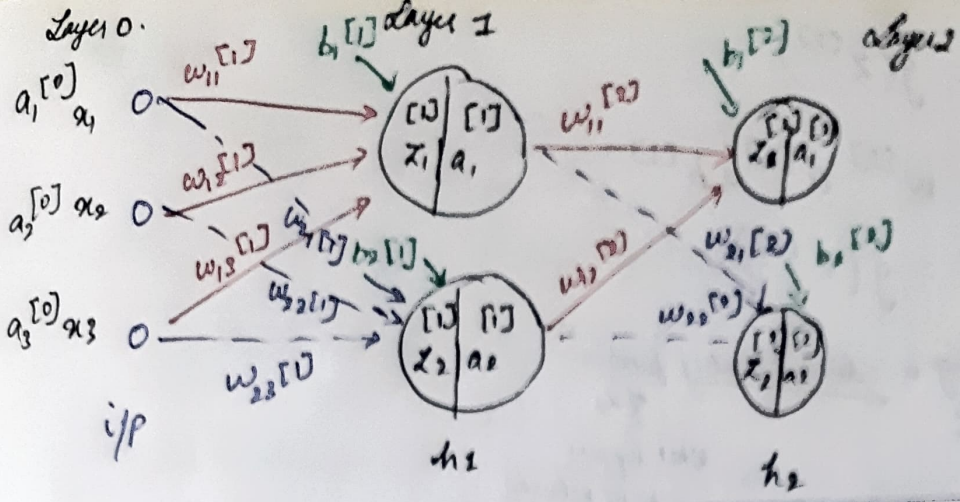
$\Rightarrow e^0$

For z_2, $\Rightarrow w_{21}a_1 + w_{22}x_2$

$= 0.01 \times 0 + 0.01 \times 0$

$= 0.$

$a_1 = Softmax(z_1) \Rightarrow \frac{e^{z_1}}{\Sigma e^{z_1} + e^{z_2}} \Rightarrow \frac{e^0}{\Sigma e^0 + e^0} = \frac{1}{2}$



$w^{[l]}$ – connections → matrix

b – bias → vector

$z = w() + b$

$a^{[l]} = g(z)$ → element-wise operation.

$a^{[0]} = x$

w: ② Rows – No. of neurons in the l^{th} layer

③ Cols – No. of neurons in $(l-1)^{th}$ layer

z in the form of $\begin{bmatrix} z_1 \\ z_2 \end{bmatrix}_{2\times 1}$

$$z_1^{[1]} = w_{11}^{[1]} a_1^{[0]} + w_{12}^{[1]} a_2^{[0]} + w_{13}^{[1]} a_3^{[0]} + b_1^{[1]}$$

$$z_2^{[1]} = w_{21}^{[1]} a_1^{[0]} + w_{22}^{[1]} a_2^{[0]} + w_{23}^{[1]} a_3^{[0]} + b_2^{[1]}$$

$$\begin{bmatrix} w_{11}^{[1]} & w_{12}^{[1]} & w_{13}^{[1]} \\ w_{21}^{[1]} & w_{22}^{[1]} & w_{23}^{[1]} \end{bmatrix}_{2\times 3} \qquad b^{[1]} = \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \end{bmatrix}_{2\times 1} \text{ vector}$$

$$z_1^{[2]} = w_{11}^{[2]} a_1^{[1]} + w_{12}^{[2]} a_2^{[1]} + b_1^{[2]}$$

$$z_2^{[2]} = w_{21}^{[2]} a_1^{[1]} + w_{22}^{[2]} a_2^{[1]} + b_2^{[2]}$$

$z^{[1]}$ for one layer $= \begin{bmatrix} z_1^{[1]} \\ z_2^{[1]} \end{bmatrix}_{2\times 1}$ vector

$2\times 3 + 2\times 1$

~~$z^{[1]} = w$~~

$$w^{[1]} = \begin{bmatrix} w_{11}^{[1]} & w_{12}^{[1]} & w_{13}^{[1]} \\ w_{21}^{[1]} & w_{22}^{[1]} & w_{23}^{[1]} \end{bmatrix}_{2\times 3}$$

$$w^{[2]} = \begin{bmatrix} w_{11}^{[2]} & w_{12}^{[2]} & w_{13}^{[2]} \\ w_{21}^{[2]} & w_{22}^{[2]} & w_{23}^{[2]} \end{bmatrix}_{2\times 3}$$

$$z^{[1]}_{2\times 1} = w^{[1]}_{2\times 3}\, a^{[0]}_{3\times 1} + b^{[1]}_{2\times 1}$$

$$a^{[1]} = \overset{\text{non-linear}}{g}\left(z^{[1]}\right) \implies z^{[2]} = w^{[2]} a^{[1]} + b^{[2]} \Rightarrow$$

$$a^{[2]} = g(z^{[2]})$$

$$z^{[3]} = w^{[3]} a^{[2]} + b^{[3]}$$

$$a^{[3]} = g(z^{[3]})$$

If $g(z) = z$ Assuming a linear / identity function of z, what happens to the network

$$a^{[3]} = g(z^{[3]})$$

$$= w^{[3]} a^{[2]} + b^{[3]}$$

$$= w^{[3]} g(z^{[2]}) + b^{[3]}$$

$$= w^{[3]} z^{[2]} + b^{[3]}$$

$$= w^{[3]} (w^{[2]} a^{[1]} + b^{[2]}) + b^{[3]}$$

$$= w^{[3]} (w^{[2]} g(z^{[1]}) + b^{[2]}) + b^{[3]}$$

$$= w^{[3]} w^{[2]} z^{[1]} + b^{[2]} + b^{[3]}$$

$$= w^{[3]} w^{[2]} (w^{[1]} a^{[0]} + b^{[1]}) + b^{[2]} + b^{[3]}$$

$$= w^{[3]} w^{[2]} w^{[1]} a^{[0]} + b^{[1]} + b^{[2]} + b^{[3]}$$

$$= \tilde{w} x + \tilde{b}$$ → linear regression.

Representation power of NN

⇒ NN with atleast one hidden layer are Universal approximators.

⇒ for any continuous fn, let's say $f(x)$, and some $\epsilon > 0$, there exists, a NN called $g(x)$ with atleast 1 hidden layer (with some non-linearity), such that

$$\forall x, \quad |f(x) - g(x)| \leq \epsilon$$

delta 3.1 or 2.9 ideally 3.

24/07/25

Parameter, Hyperparameter

Test, val → val set ⇒ to pick hyperparam

more layers, learning more complex functions → hyperparameter.

NN log construction → non-convex.. (no sure global minima)

Regularisation

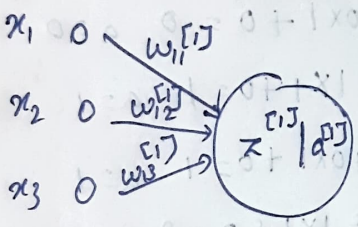
$\lambda = 0.001$ → learns efficiently

$\lambda = 0.01$

$\lambda = 0.1$

Perceptron

single layer NN with 1 neuron



$$z = w_{11}^{[1]} x_1 + w_{12}^{[1]} x_2 + w_{13}^{[1]} x_3 + b_1^{[1]}$$

$$a^{[1]} = g(z^{[1]})$$

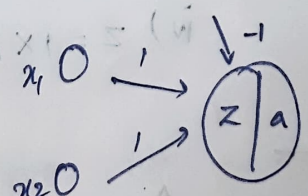
$$g = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \le 0 \end{cases}$$

First NN with 1 layer..

AND

	x_1	x_2	y
i)	0	0	0
ii)	0	1	0
iii)	1	0	0
iv)	1	1	1

single perceptron



i) $x_1 \times 1 + x_2 \times 1 + (-1) = 0 + 0 - 1$

$a \Rightarrow (-1) \Rightarrow 0$.

ii) $0 \times 1 + 1 \times 1 + (-1) = 0$

$a = 0$

iii) $1 \times 1 + 0 \times 1 + (-1) = 1 - 1 = 0$ $a = 0$

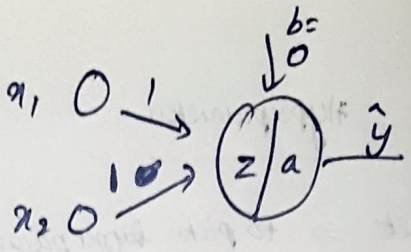
iv) $1 \times 1 + 1 \times 1 + (-1)$

$2 - 1 = 1 = 1$

$a = 1$

OR

	x_1	x_2	y
i)	0	0	0
ii)	0	1	1
iii)	1	0	1
iv)	1	1	1



i) $z = 1\times0 + 1\times0 + 0 = 0$ $a = 0$

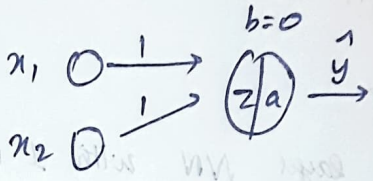
ii) $z = 0\times1 + 1\times1 + 0 = 1$ $a = 1$

iii) $z = 1\times1 + 0\times1 + 0 = 1$ $a = 1$

iv) $z = 1\times1 + 1\times1 + 1 = 3$ $a = 1$

XOR

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0



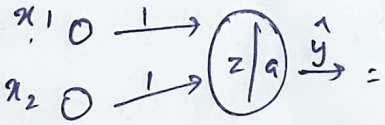
i) $z = 0\times1 + 0\times1 + 0 = 0$ $a = 0$

ii) $z = 0\times1 + 1\times1 + 0 = 1$ $a = 1$

iii) $z = 1\times1 + 0\times1 + 0 = 1$ $a = 1$

iv) $z = 1\times1 + 1\times1 + 0 = 2$ $a = 1$

$b = -1$



=

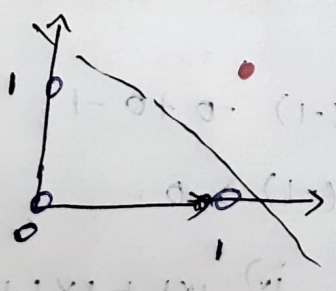
i) $z = 0\times1 + 0\times1 - 1 = -1$ $a = 0$

ii) $z = 0\times1 + 1\times1 - 1 = 0$ $a = 0$

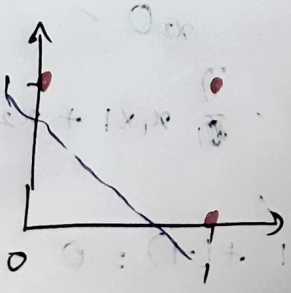
iii) $z = 1\times1 + 0\times1 + (-1) = 0$ $a = 0$

iv) $z = 1\times1 + 1\times1 + (-1) = 1$ $a = 1$

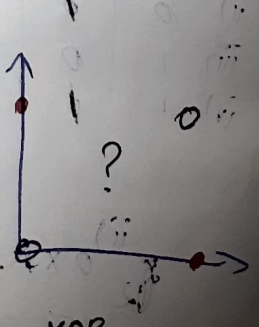
Decision boundaries



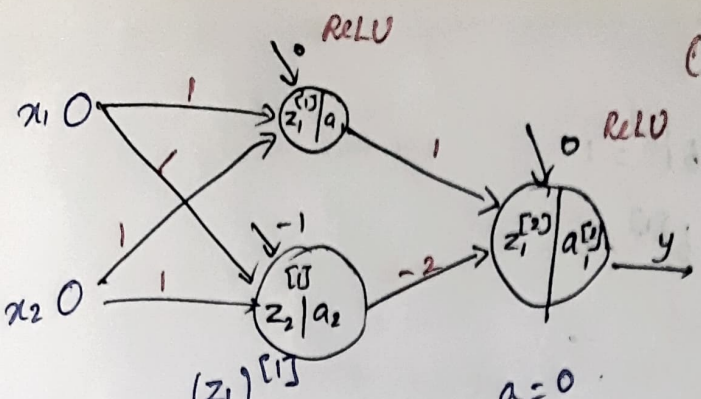
AND



OR



XOR



(Sigmoid / Softmax will not work)

XOR

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

$(z_1)^{[1]}$

i) $0\times1+0\times1+(0)=0 \rightarrow a=0$

ii) $0\times1+1\times1+(0)=1 \rightarrow a=1$

iii) $1\times1+0\times1+(0)=1 \rightarrow a=1$

iv) $1\times1+1\times1+(0)=2 \rightarrow a=2$

$z_1 = 4$

$a_1 = \max(0,4) = 4$

$$z=\begin{bmatrix}4\\0\end{bmatrix} \quad a=\begin{bmatrix}4\\0\end{bmatrix}$$

$(z_2)^{[1]}$

i) $0\times1+0\times1+(-1)=-1 \quad a=0$

ii) $0\times1+1\times1+(-1)=0 \rightarrow a=0$

iii) $1\times1+0\times1+(-1)=0 \rightarrow a=0$

iv) $1\times1+1\times1+(-1)=1 \rightarrow a=1$

$z_1 = 0$

$a_2 = \max(0,0) = 0$

2nd layer $\rightarrow 0\times1+0\times-2=0$

2nd layer $\rightarrow 1\times1+0\times-2=1$

2nd layer $\rightarrow 0\times1+0\times-2=1$

2nd layer $\rightarrow 2\times1+1\times-2=0$

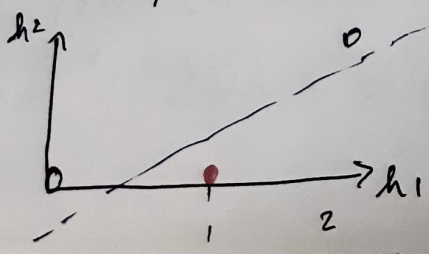
2nd layer

$z_1^{[2]} = 4\times1 + (-2)\times0 + 0 = 4$

$a_1^{[2]} = 4$

The original data is transformed into a new space. → where we can find the decision boundary

The hidden representation h



Weight initialization

i) $W_{ij} \sim \text{Uniform}([-0.1, 0.1])$ → a better way than 0.

ii) $W_{ij} \sim N\left(0, \frac{2}{\sqrt{n^{[l]} + n^{[l-1]}}}\right)$ → Xavier initialization
also commonly used (refer papers)

n - neurons in the layer.

try both approaches → good enough.
(one is not better than the other . try both)

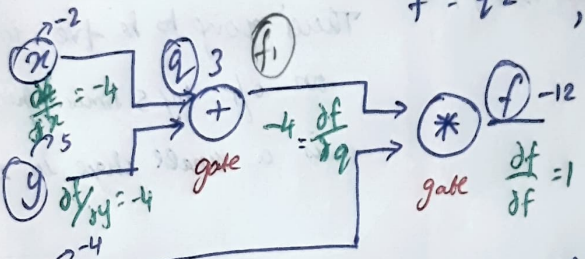
Weight init
⇒ helps converge faster
⇒ learns better

We can also take Random variance, we can also introduce weights depends on domain knowledge

Computational Graph → Datastructure

$f = (x+y)z$; Let $q = x+y$, $\frac{\partial q}{\partial x} = 1$, $\frac{\partial q}{\partial y} = 1$

$f = qz$, $\frac{\partial f}{\partial q} = z$, $\frac{\partial f}{\partial z} = 9$



ckt → 3 = $\frac{\partial f}{\partial z}$ → unit change in z, 3 units change in f. $\frac{\partial f}{\partial x}$, $\frac{\partial f}{\partial y}$ & $\frac{\partial f}{\partial z}$.

$\frac{\partial f}{\partial x} = \frac{\partial q}{\partial x} \cdot \frac{\partial f}{\partial q} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial x}$

change of f wrt x

global gradient

local gradient

given a gate we can compute a local gradient

$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial y}$ → $-4 \times 1 = -4$

$\frac{\partial f}{\partial z}$

local gate influence × global gate influence

29/07/25

$$f(x,y,z) = (x+y)z$$

$$q = (x+y)$$

Wherever there's an arithmetic ops, put a gate.

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial y}$$

$$= 1 \cdot z$$

$$= z$$

$$= -4$$

Example:

$$f(x,y,z) = x + yz\ q$$

$x = -2$

$y = 5$

$z = -4$

local x global

$$\frac{\partial f}{\partial z} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial z}$$

$$= 1 \cdot y$$

$$= y$$

$$= 5$$

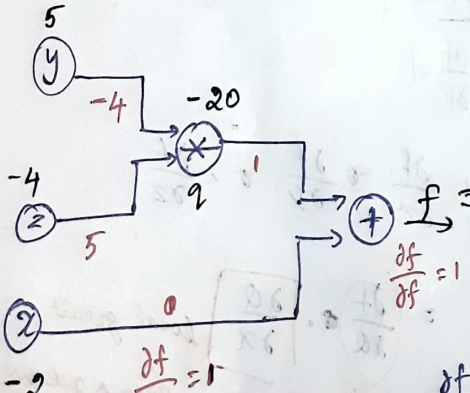
① Identify additional functions

② Draw a computational graph

③ Perform forward pass

④ Perform backward pass starting from the end of the ckt.

There's going to be +ve impact on o/p by 5 times when there is a small change in z



$$\frac{\partial y}{\partial q} = 1 \quad \frac{\partial f}{\partial q} = 1$$

$$\frac{\partial f}{\partial f} = 1 \quad \frac{\partial f}{\partial x} = 1 \quad \frac{\partial f}{\partial y} = z, \quad \frac{\partial f}{\partial z} = y$$

$$\frac{\partial y}{\partial f} = \frac{\partial y}{\partial q} \cdot \frac{\partial q}{\partial f}$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial f} \cdot \frac{\partial f}{\partial x} = 1 \times 1 \Rightarrow 1$$

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0 + w_1x_1 + w_2)}}$$

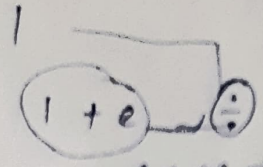
$w_0 = 2$

$x_0 = -1$

$w_1 = -3$

$x_1 = -2$

$w_2 = -3$



$e^{-(w_0x_0 + w_1x_1)}$

$f_1 = w_0x_0$

$f_2 = w_1x_1$

$f_3 = f_1 + f_2 + w_2$

$f_4 = -f_3$

$f_5 = e^{f_4}$

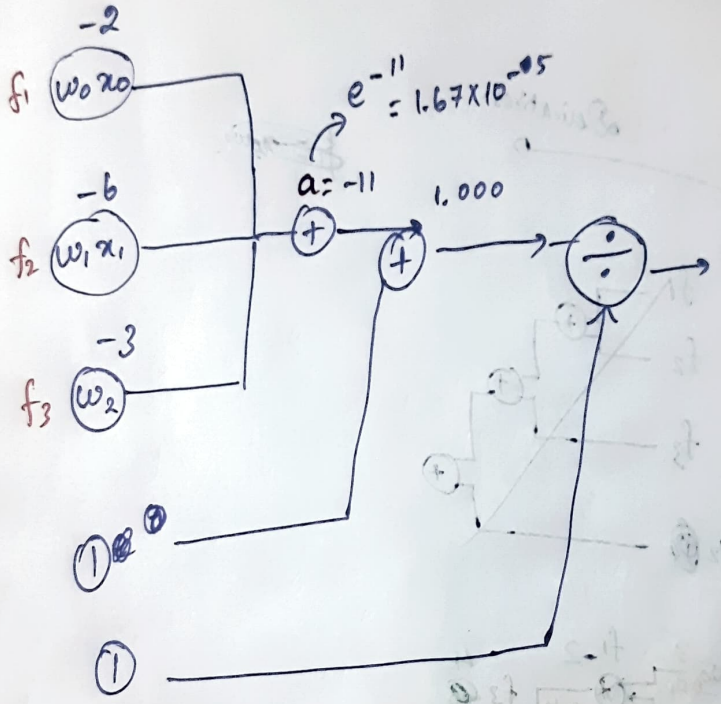
$f_6 = 1 + f_5$

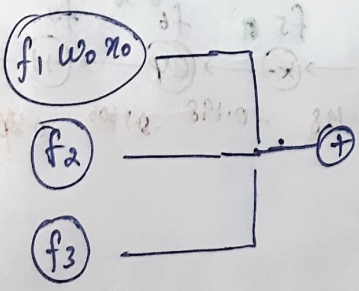
$f_7 = 1/f_6$

$w_0x_0 = f_1$

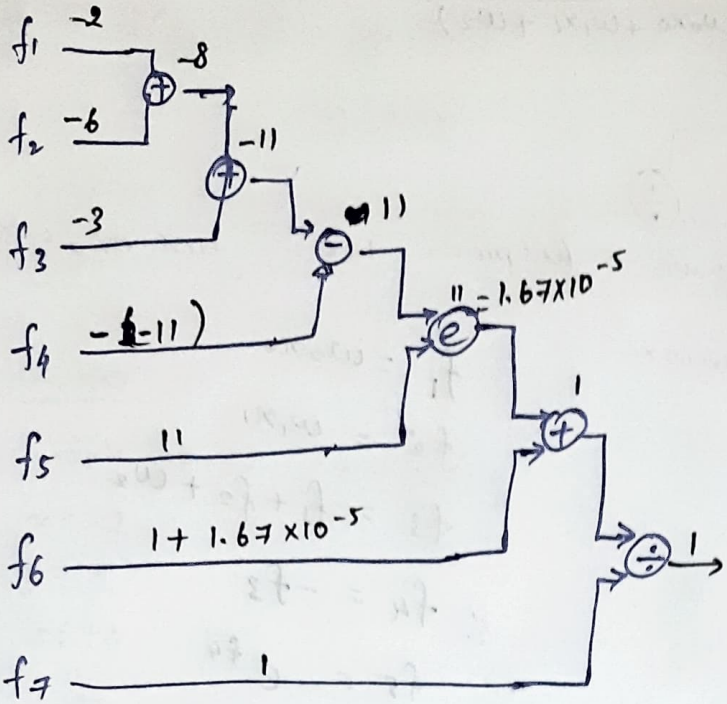
$w_1x_1 = f_2$

$w_2 = f_3$





keep it into 2.



↓

break down to 2 inputs.

Derivatives.

$f_1 = w_0 x_0$

$f_2 = w_1 x_1$

$f_3 = f_1 + f_2$

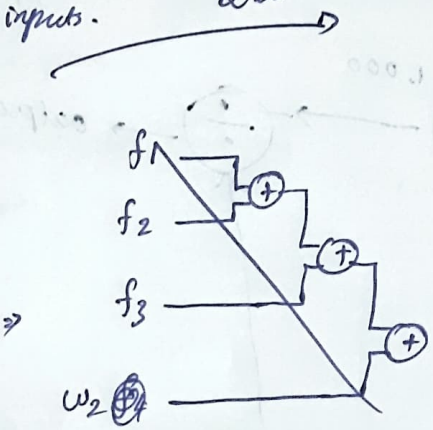
$f_4 = f_3 + w_2$

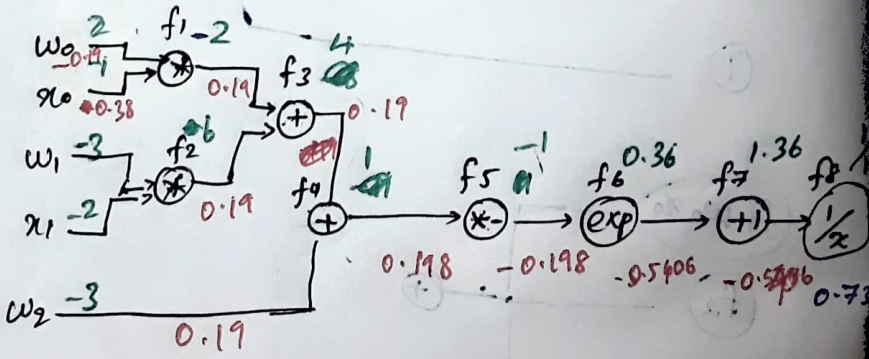
$f_5 = -f_4$

$f_6 = e^{f_5}$

$f_7 = 1 + f_6$

$f_8 = 1/f_7$





gradients

local × global

$\frac{df_8}{df_8} = 1 \quad f_1 = \frac{\partial f_1}{\partial w_0} = x_0 \quad \frac{\partial f_1}{\partial x_0} = w_0$

$f_2 = \frac{\partial f_2}{\partial w_1} = x_1 \quad \frac{\partial f_2}{\partial x_1} = w_1$

Local gradients

$\frac{\partial f}{\partial x_0}$ $\frac{\partial f}{\partial w_1}$ $\frac{\partial f}{\partial x_1}$ $\frac{\partial f}{\partial w_2}$

$f_3 = f_1 + f_2 \Rightarrow \frac{\partial f_3}{\partial f_1} = 1 \; ; \frac{\partial f_3}{\partial f_2} = 1$

$f_4 = f_3 + w_2 \Rightarrow \frac{\partial f_4}{\partial f_3} = 1 \; ; \frac{\partial f_4}{\partial w_2} = 1$

$f_5 = -f_4 \Rightarrow \frac{\partial f_5}{\partial f_4} = -1 \Rightarrow -1 \times -0.198$

$f_6 = e^{f_5} \Rightarrow \frac{\partial f_6}{\partial f_5} = e^{f_5} \Rightarrow e^{-1} \Rightarrow \times$ Global -0.5406

$\rightarrow -0.198$

$f_7 = 1 + f_6 = \frac{\partial f_7}{\partial f_6} = 1 \times f_6 = -0.5406$

$\frac{\partial f_8}{\partial f_5} = \left(\frac{\partial f_7}{\partial f_6} \times \frac{\partial f_8}{\partial f_7}\right)$

$f_8 = \frac{1}{f_7} \Rightarrow (f_7)^{-1} \Rightarrow \frac{\partial f_8}{\partial f_7} = (-f_7)^{-2} \Rightarrow \frac{-1}{f_7^2}$

$\Rightarrow -\frac{1}{(.36)^2}$

$= -0.5406$

$\left(\frac{\partial f_8}{\partial f_7} \times \frac{\partial f}{\partial f_8}\right)$

$\frac{\partial f_8}{\partial f_7}$ = local × global

$= -0.5406 \times 1$

$= -0.5406$

$\frac{\partial f_8}{\partial f_6}$ = local × global

$= \frac{\partial f_7}{\partial f_6} \times \frac{\partial f_8}{\partial f_7}$

$= 1 \times -0.5406$

$= -0.5406$

$\frac{\partial f_8}{\partial f_5} = \frac{\partial f_6}{\partial f_5} \times \frac{\partial f_8}{\partial f_6}$

$= e^{-1} \times -0.5406$

$\Rightarrow -0.198$

$\frac{\partial f_8}{\partial f_4} = \frac{\partial f_5}{\partial f_4} \times \frac{\partial f_8}{\partial f_5}$

$= -1 \times -0.198$

$= 0.198$

$\frac{\partial f_8}{\partial f_3} = \frac{\partial f_4}{\partial f_3} \times \frac{\partial f_8}{\partial f_4}$

$= 1 \times 0.198$

$$\frac{\partial f_8}{\partial w_2} = \frac{\partial f_4}{\partial w_2} \times \frac{\partial f_8}{\partial f_4} \rightarrow 1 \times 0.19$$

$$\rightarrow 0.19$$

$$\frac{\partial f_8}{\partial f_3} = \frac{\partial f_4}{\partial f_3} \times \frac{\partial f_8}{\partial w_2} \rightarrow 1 \times 0.19$$

$$= 0.19.$$

$$\frac{\partial f_8}{\partial f_2} = \frac{\partial f_3}{\partial f_2} \times \frac{\partial f_8}{\partial f_3}$$

$$\rightarrow 1 \times 0.19$$

$$= 0.19$$

$$\frac{\partial f_8}{\partial f_1} = \frac{\partial f_2}{\partial f_1} \times \frac{\partial f_8}{\partial f_2}$$

$$= 1 \times 0.19$$

$$\frac{\partial f_1}{\partial w_0} \frac{\partial x_0}{\partial f_1} = \times 0.19 \quad \frac{\partial x_0}{\partial f_1} \times \frac{\partial f_8}{\partial f_1}$$

$$\frac{\partial f_8}{\partial w_0} = \frac{\partial f_1}{\partial w_0} \times 0.19 = x_0 \times 0.19 = -1 \times 0.19 = -0$$

$$\frac{\partial f_8}{\partial x_0} = \frac{\partial f_1}{\partial x_0} \times 0.19 = w_0 \times 0.19 = 2 \times 0.19 = 0.3$$

$$\frac{\partial f_8}{\partial w_1} =$$

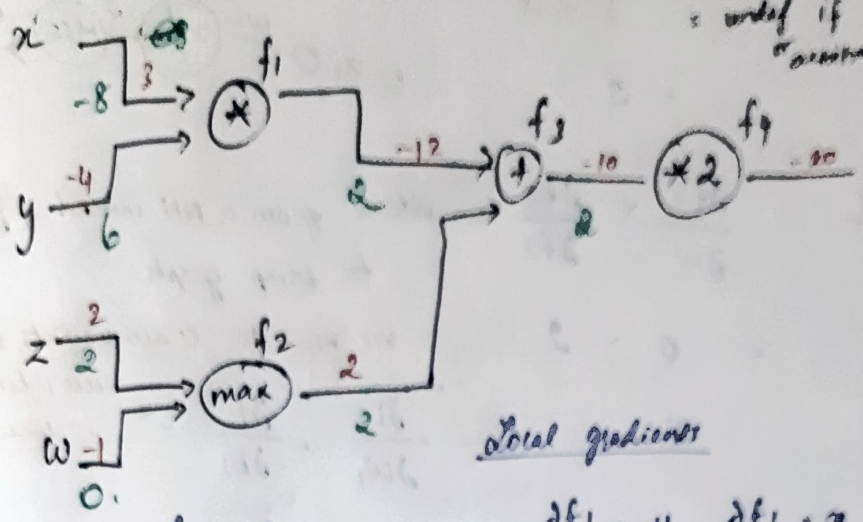
$$\frac{\partial f_8}{\partial x_1} =$$

$\frac{\partial f_2}{\partial z} = 1$ if $w > z$

$= 0$ if $w < z$

$=$ undef if $w = z$

"assume"



Local gradients

$f_1 = xy$: $\frac{\partial f_1}{\partial x} = y$ $\frac{\partial f_1}{\partial y} = x$

$f_2 = \max(z, w)$: 1

$f_3 = f_1 + f_2$: $\frac{\partial f_3}{\partial f_1} = 1$ $\frac{\partial f_3}{\partial f_2} = 1$

$f_4 = 2 \times f_3$: $\frac{\partial f_4}{\partial f_3} = 2$

$\frac{\partial f}{\partial f} = 1$

$\frac{\partial f}{\partial f_4} \times \frac{\partial f_4}{\partial f_3} = \frac{\partial f_4}{\partial f_3} \times 1$

$= 2 \times 1 = 2$

$\frac{\partial f_4}{\partial f_1} = \frac{\partial f_3}{\partial f_1} \times \frac{\partial f_4}{\partial f_3}$

$= 1 \times 2 \rightarrow ?$

$\frac{\partial f_4}{\partial x} = \frac{\partial f_1}{\partial x} \times \frac{\partial f_4}{\partial f_1}$

$= y \times 2$

$= -4 \times 2 = -8$

$\frac{\partial f_4}{\partial y} = \frac{\partial f_1}{\partial y} \times \frac{\partial f_4}{\partial f_1}$

$= x \times 2$

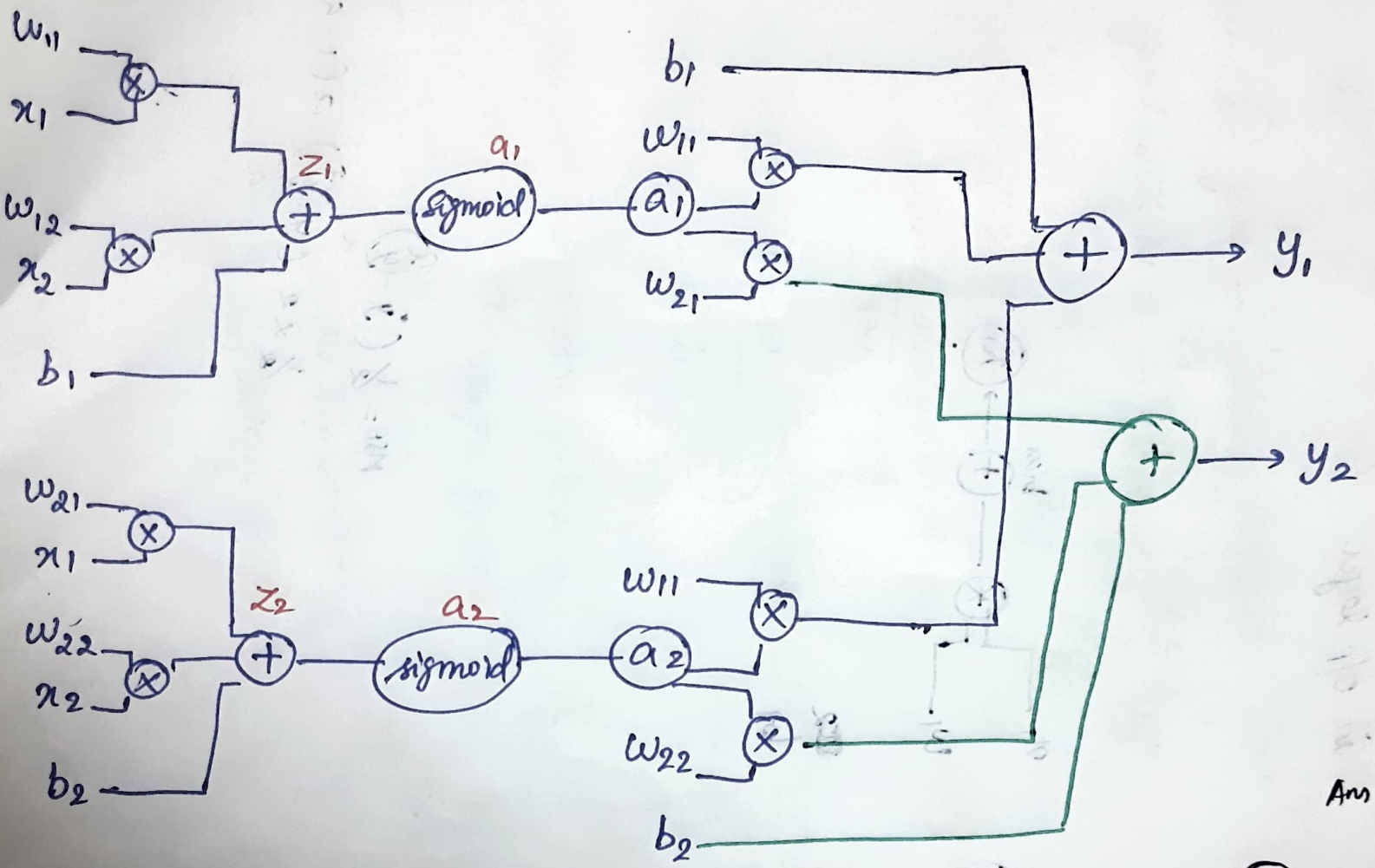
$= 3 \times 2$

$= 6$

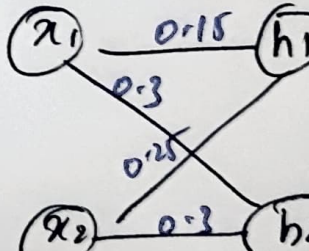
$\frac{\partial f_4}{\partial f_2} = \frac{\partial f_3}{\partial f_2} \times \frac{\partial f_4}{\partial f_3}$

$= 1 \times 2$

$\frac{\partial f_4}{\partial f_2} = 2$


w_{11}
x_1
w_{12}
x_2
b_1
z_1
sigmoid
a_1
b_1
w_{11}
a_1
w_{21}
y_1
w_{21}
x_1
w_{22}
x_2
b_2
z_2
sigmoid
a_2
w_{11}
a_2
w_{22}
b_2
y_2

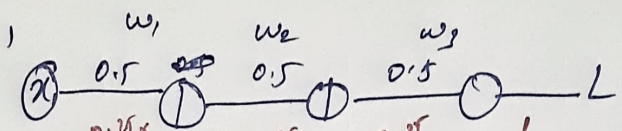
Ans:

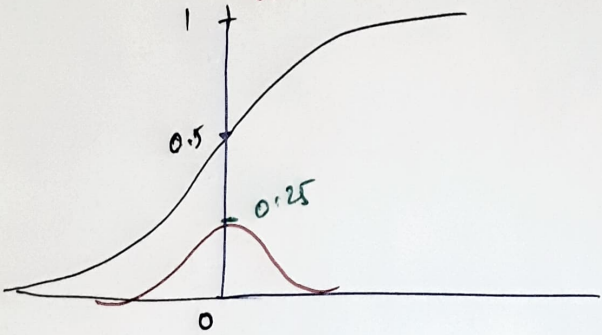

x_1
0.15
0.3
0.25
x_2
0.3

04/08/25

Vanishing & Exploding gradient problems

Vanishing gradient problem





→ The max value that the sigmoid fn can take is 0.25

↓

Gradients get diminished.

↓

1st layer becomes almost 0

↓

no learning.

⇒ Sigmoid & tanh has vanishing gradient problem ⇒ Key problem in sigmoid

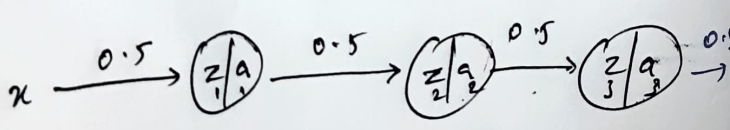
↓

We use ReLU

But ReLU → exploding gradient problem. ⇒ can be avoided by thresholding / clipping the gradi

example

Assume all activn fnct are sigmoid



$z_1 = 1 \times 0.5 = 0.5$

$a_1 = \sigma(z_1) = 0.622$ $\frac{1}{1+e^{-z_1}} = \frac{1}{1+0.606} \Rightarrow 0.625$ a_1

$z_2 \Rightarrow 0.625 \times 0.5 = 0.312$

$\frac{1}{1+e^{-0.3125}} \neq \frac{1}{1.731}$

$\Rightarrow 0.57$ a_2

$L_0 = \frac{1}{2}(a_3 - y)^2 = 0.572$

$\frac{\partial L}{\partial a_3} = \frac{1}{2} \times 2 (a_3 - y) \cdot 1$

$z_3 \Rightarrow 0.571 \times 0.5 \Rightarrow 0.288$

$a_3 \Rightarrow \frac{1}{1+e^{-0.288}} \Rightarrow \frac{1}{1.749} \Rightarrow 0.571$

$$\frac{\partial L}{\partial a_3} = a_3 - y \Rightarrow 0.572 - 1$$

$$\boxed{\frac{\partial L}{\partial a_3} = -0.428}$$

Layer 3: L3,
Grad = -0.107

Layer 2
Grad = -0.013

Layer 1
Grad = -0.0015

Explody gradient

w = 5

L1

$z_1 = w \cdot x = 5 \cdot 1 = 5$

$a_1 = \text{ReLU}(z_1) = 5$

L2

$z_2 = 5 \cdot a_1 = 5 \cdot 5 = 25$

$a_2 = \text{Relu}(z_2)$

$= 25$

$$\frac{\partial L}{\partial a_2} = \frac{\partial a_3}{\partial a_2} \cdot \frac{\partial L}{\partial a_3}$$

$= 0.25 \times -0.428$

$= -0.107$

$g(z) = a(1-g(z))$

$a_3 \quad (1-a_3)$

$0.5(1-0.5)$

$0.5(0.5)$

0.25

$L = \frac{1}{2}(a_3 - y)^2$

$\frac{\partial L}{\partial a_2} = \frac{1}{2} \times 2(a_3 - y) \Rightarrow 1$

$\frac{\partial a_3}{\partial a_2} =$

$$\frac{\partial L}{\partial a_1} = \frac{\partial a_2}{\partial a_1} \cdot \frac{\partial L}{\partial a_2}$$

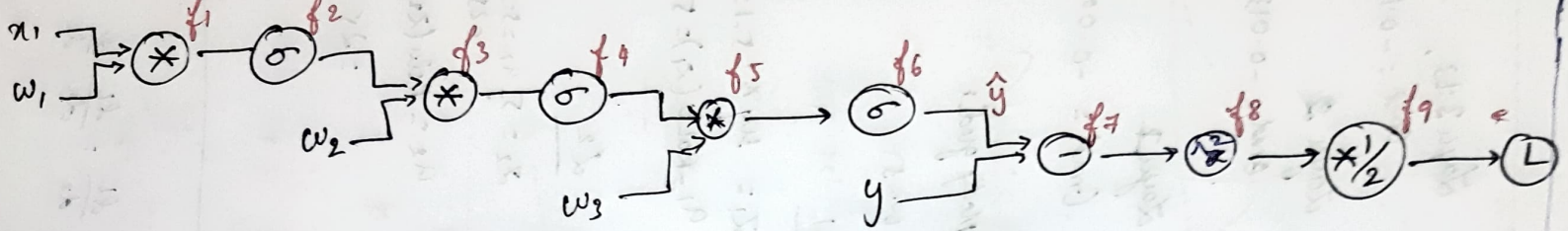
$= a_2(1-a_2) \times -0.107$

$= 0.57(1-0.57) \times -0.107$

$\Rightarrow$

$0.57(1-0.57)$

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z}$$



$f_1 = x \cdot w_1$

$f_2 = \sigma f_1 \Rightarrow \frac{\partial f_2}{\partial f_1} = f_1(1-f_1)$

$f_3 = w_2 \cdot f_2 \quad \frac{\partial f_3}{\partial f_2} =$

$f_4 = \sigma f_3$

$f_5 = w_3 \cdot f_4$

$f_6 = \sigma f_5$

~~$\hat{y} = f_6 \cdot y$~~

$f_7 = f_6 - y$ or $y - f_6$

$f_8 = \cdot z(f_7)$

4

$\frac{\partial L}{\partial z_3} = \frac{\partial L}{\partial a_3} \cdot Relu(z_3)$

$= 125 \cdot 1 = 125$

z_2

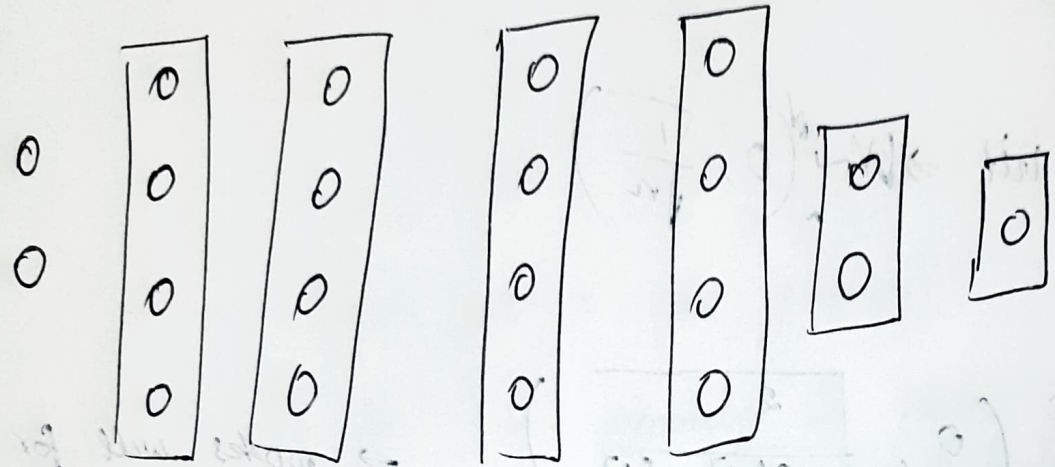
$\frac{\partial L}{\partial z_2} = 625$

$\frac{\partial L}{\partial z_3} = 3125$

06/08/25

BatchNorm

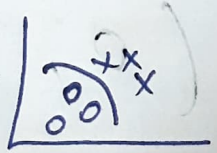
Batchnorm is applied in every layers

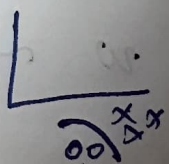


- Normalize each layer
- Apply it to z.

There is a possibility that the data can get shifted or scaled.

It may not converge





Scale and shift operation → firstly on the data

(like bias term in ML → where we shift)

↓

But how much to shift and scale?

↓

obtain the learnig parameter → which is learnt from the data.

(In the initial layers, the shift still happens very minimally)

⇒ Scale & shift → γ & β respectively

not hyperparameters, learnable parameters.

⇒ $\gamma^{[l]}$ $\beta^{[l]}$

⇒ Apply it for every batch

eg: z from some layer

$z = [4, 8, 6, 10]$

↓

z - output of a hidden layer

$$\mu = \frac{4+8+6+10}{4+\epsilon(10^{-7})} = 7 \longrightarrow \text{for avoidy undefined error}$$

$$\sigma = \frac{1}{4}\sum_{i=1}^{4}(z_i - \mu)^2$$

$$\sigma = \frac{1}{4}\left[(4-7)^2 + (8-7)^2 + (6-7)^2 + (10-7)^2\right]$$

$$\sigma = 5$$

$$\hat{z} = \frac{z-\mu}{\sigma}$$

$$\tilde{z} = \gamma\hat{z} + \beta$$

Pass this $\hat{z}$ to the function $\tilde{z} = \gamma\hat{z} + \beta$

$$z = \frac{x-\mu}{\sigma+\epsilon}$$

* BatchNorm only for hidden layers

⇒ β & γ are learnable parameters

$\gamma = \sigma$, $\beta = 1$

$\tilde{z}_1 = z(-1.3416) + 1$

$= -1.6832$

$\tilde{z}_2 = 1.8944$

$\tilde{z}_3 = 0 \ldots$

W & γ & β are also getting updated.

(Whatever rules we have for parameters holds good for γ & β)

⇒ weights are getting updated every batch.

⇒ How many samples to take in batch is a hyperparameter?

⇒ Size of the batch impact on performance?

↓

time taken to converge & when it is stuck

32, 64, 128, 256

every neurons are scaled by p. (as it only contributed 'p' times).

⇩

STANDARD DROPOUT · [Test time complexity ↑]

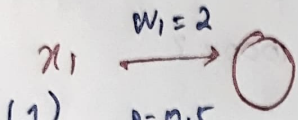
Inference time complexity shd be extremely low ⇩.

↓

∴ Inverted dropout has been introduced.

↳ no extra computn during test time (only which reduces time)
↳ keep all neurons (no drops)
↳ scale by 1/p during train time

Illustn



↓

keep-prob

TRAIN

Sample 1 → o/p = 2
Sample 2 → ⊗ dropout → o/p = 0
Sample 3 → ⊗ dropout → o/p = 0
Sample 4 → o/p = 2

Total o/p = 4

↓

$4 \times \frac{1}{0.5} = 8$

TEST

Sample 1 → o/p = 2
Sample 2 → o/p = 2
Sample 3 → o/p = 2
Sample 4 → o/p = 2

Total o/p = 8

We didn't do anything at test bcoz during the train tweak to get same o/p ? How?

↓ raw o/p

Scale by 1/p to get the total o/p

^

For each neuron, multiply by 1/p → in forward pass itself.

L2 & dropout (pytorch → nn.dropout ( ))

↓ commonly used.

↓ for every layer

Apply dropout & scaling before activation

$z = wx + b$

↓

Dropout mask: $\tilde{z} = M \cdot z$ → Scale the remaining activ neurons: $\tilde{z}_{scaled} = \frac{z}{p}$

↓ mask

$a = f(\tilde{z}_{scaled})$

Convolutional Neural Network

Starts w computer vision problems

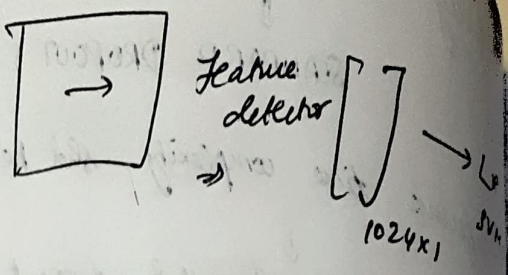
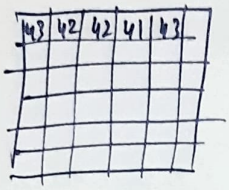


Image - matrix where every cell contains a pixel value

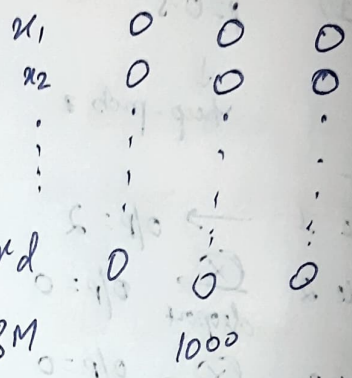
CNN uses example: Images from g... & Medium.

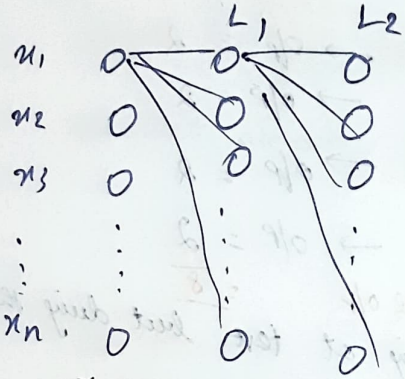
We can use deep NN,

give inputs as the pixel value



1000 x 1000 x 3





$W[1] = 3M \times 100 \Rightarrow$ 3 billion.

Feature extraction techniques in CV:

⇒ Histogram of Oriented Gradients HOG

⇒ Histogram of Oriented flows (HOF)

⇒ Optical flow feature.

Most of the real world images:

⇒ the color transition / gradient

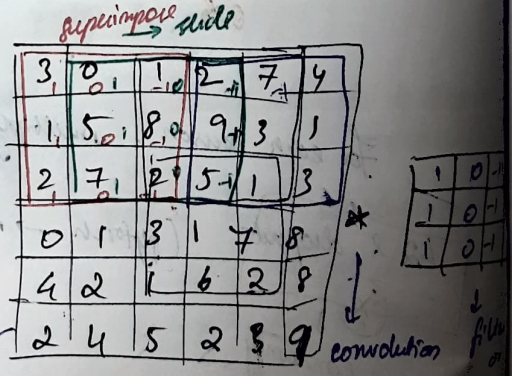
⇒ inherent property of an image is the smooth transition

Heterogeneity helps in detecting the image.

Heart of CNN

Convolution Operation

Fundamental operation in CNN.



$\Rightarrow 3\times1 + 0\times0 + 1\times-1 + 1\times1 + 5\times0 + 8\times-1 + 2\times1 + 7\times0 + 2\times-1$

$\Rightarrow 3 - 1 + 1 - 8 + 2 - 2$

$= -5$

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

→ $-2 + 5 - 9 + 7 + 5$

$= -11 + 7$

$= -4$

→ $1 + 0 + (-7) + 8 - 3 + 2 + 1$

→ $10 - 10$

⇒ 0

→ $2 - 1 + 3 - 7 + 1 - 2$

-4

⇒ Square matrix (mostly)

Vertical & horizontal

* Every filters are going to give me features (edges)

How many filters, what filters - hyper parameters

13/08/25

① Convolution ops.

learning process : learning the weights

instead of raw pixel values

↓

edges to be the features.

Sobel edge detection

Canny edge detection

Laplacian edge detection

Prewitt edge detection

Scharr edge detection.

Handcrafting features didn't work well.

edges as features

↓

(after detection techniques)

↓

SVMs

Convolution operation

Vertical edge detection

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

$*$

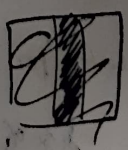
1	0	-1
1	0	-1
1	0	-1

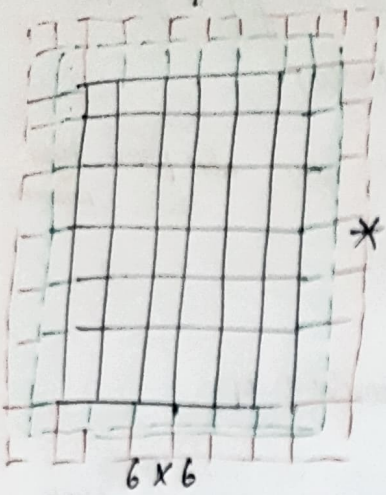
$=$

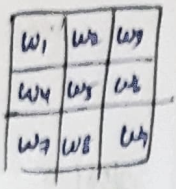
0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

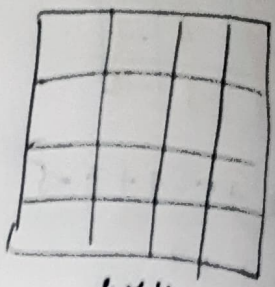
Horizontal edge detection

1	1	1
0	0	0
-1	-1	-1









$[n-f+1 \times n-f+1]$

n + paddij = 1 + paddij 2

o/p image size = $n-f+1 \times n-f+1$

Image gets shrunk → when deep layers, goes through the i/p will get shrunk (vanish)

② problems PADDING can ...

① Shrinking problem

② Equal attention to all pi...

Neurons have does - convolution ops (instead of wx+b).

⇒ The filter doesn't see ~~the~~ all the boxes the same no. of times.

eg: 7×7 matrix + paddij

9-3+1 ⇒ 7×7 matrix

p = 1 extension by 1 on all 4 sides
paddij = 1

Paddij is done to keep the same image.

0 is okay to paddij, copying the 1st rows & columns also doesn't change so much.

⇒ 0 is most commonly used for paddij

[No random numbers]

$n+2p-f+1 \times n+2p-f+$

~~0~~ n

Valid convolution: i/p size ≠ o/p size

- paddij value - hyperparameter.

p = 1 or 2 most commonly..

Same convolution: i/p = o/p size

p can be different for diff layers.

6×6 i/p * [3×3] = size of o/p paddij p = 1

~~7 - 3 + 1 = 5 × 5~~ [2 columns & rows are added]

8 - 3 + 1 ⇒ 6×6

14/08/25

Theory

Striding + Padding

2	3	7	4	6
6	6	7	8	7
3	4	8	3	8
7	8	3	6	6
4	2	1	8	3

$*$

1	-1
0	-1

$= ?$

$f = 2 \times 2$

$n = 5 \times 5$

$p = 1$, $s = 2$

$p = 1$

0	0	0	0	0	0	0
0	2	3	7	4	6	0
0	6	6	7	8	7	0
0	3	4	8	3	8	0
0	7	8	3	6	6	0
0	4	2	1	8	3	0
0	0	0	0	0	0	0

-2	-7	-6
-9	-11	-7
-11	4	-3

3×3

p:

$0 \times 1 + 0 \times -1 + 0 \times 0 + -1 \times 2 \Rightarrow -2$

p

$-6 - 3 = -9$

$6 - 9 - 8 \quad 6 - 17 = -11$

$8 - 7 + 0 - 8 = -7$

$0 - 7 - 4 = -11$

$8 - 3 + 0 - 1 = 4$

$6 - 6 + 0 - 3 \Rightarrow -3$

$n = 5 \times 5 \quad f = 2 \times 2 \quad 4 \times 4 = -$

$n = 5 \times 5 \quad f = 2 \times 2$

$p = 1 \quad s = 2$

$5 + 2(1) - 2 + 1$

$5 + 1 - \quad = 3$

$6 - x = 3$

$6 - (\quad) = 3$

or $6/\quad = 3$

$6/3 = 3$

formula for output

$$n = \frac{n + 2p - f + 1}{s}$$

$$5 = \frac{5 + 2(1) - 2 + 1}{s}$$

$$5 = \frac{5+1}{s} \Rightarrow \frac{6}{s} = 3$$

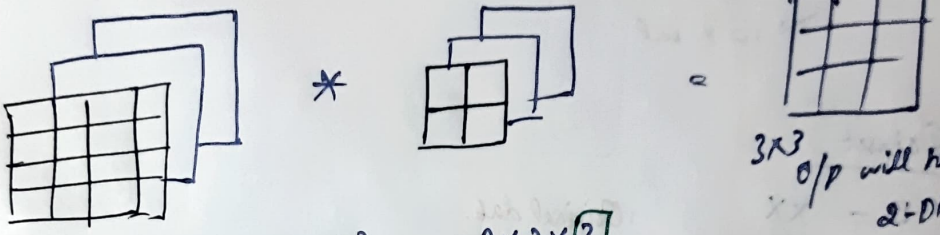
$$s = 2$$

with striding & padding

$$n \times n = \left\lfloor \frac{n+2p-f+1}{s} \right\rfloor \times \left\lfloor \frac{n+2p-f+1}{s} \right\rfloor$$

⇒ floor value.

Convolutions on a 3D vector



(3 images stack) 2×2×3

3×3 o/p will be 2-Dimensional flattened 2-D matrix

4×4×3 → identical

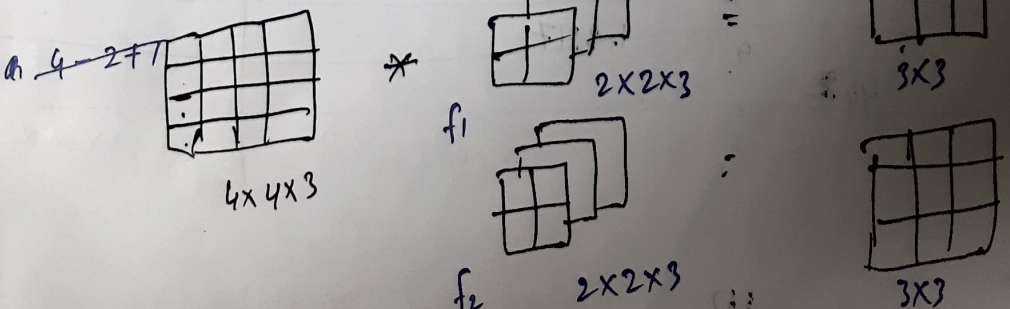
H×W× channel

channels in i/p = # filters

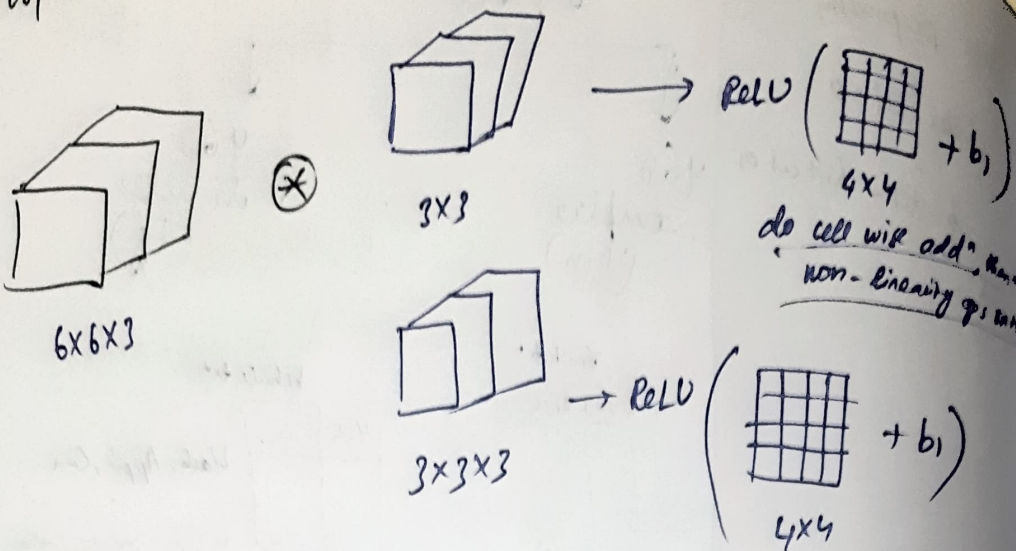
Superimpose the cuboids to the correspondg image channel

sum up the values for the

(only 1 channel can have info or 2, can be anything) eg: RGB → green channel has more information (feature)



14/08/25



do cell wise addⁿ, then non-linearity ftn is taken

do for in ANN, we had

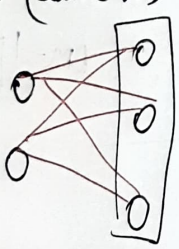
$$z^{[l+1]} = W^{[l+1]} a^{[l]} + b^{[l+1]}$$

$$a^{[l+1]} = ReLU\left(z^{[l+1]}\right)$$

The filters are applied only in the portion of the image at a time ⇒ (same wts across whole image - i) weight sharing)

ii) we look into only some part of image - sparsity of connection

Analogy:



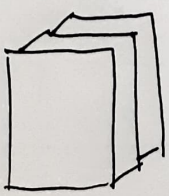
diff weights for every connection from 1 neuron to next

↓ replaced by Conv layers

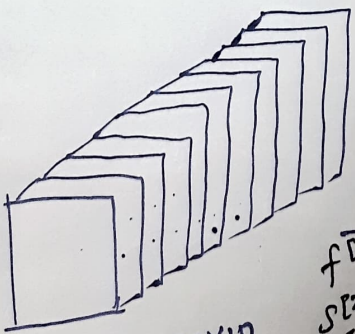
these 2, keeps parameters to less in number

No. of parameters will become significantly less, just when we do the weight sharing.

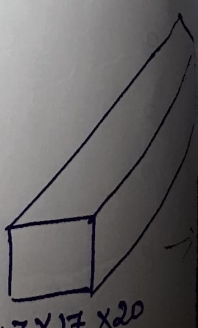
Conv Net



$f^{[1]}=3$
$s^{[1]}=1$
$p^{[1]}=0$
10 filters



$f^{[2]}=5$
$s^{[2]}=2$
$p^{[2]}=0$
20 filters

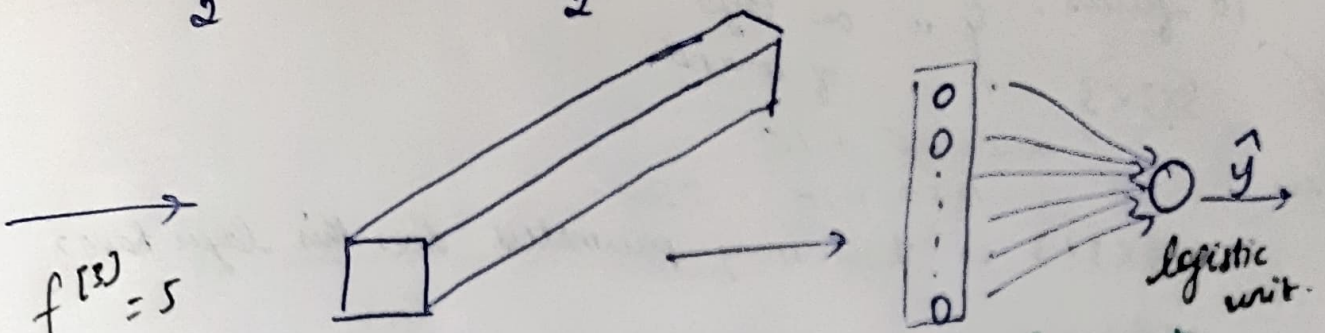


$4-2+1 \Rightarrow 3\times3$

$\frac{n+2p-f}{s}+1 \Rightarrow \frac{3+2(0)-3}{1}+1 \Rightarrow \frac{3+2(0)-3}{1}+1 \Rightarrow 1$

$36+1 \Rightarrow 37\times37$

$$\frac{37+2(0)-5}{2}+1 = \frac{32}{2}+1 = 16+1 \Rightarrow 17\times17\times10$$



$f^{[1]} = 5$

$s^{[1]} = 2$

$p^{[1]} = 0$

40 filters

$7\times7\times40 = 1960$ px.

1960 flatten out

No. of channels are increasing

40 diff types of feature.

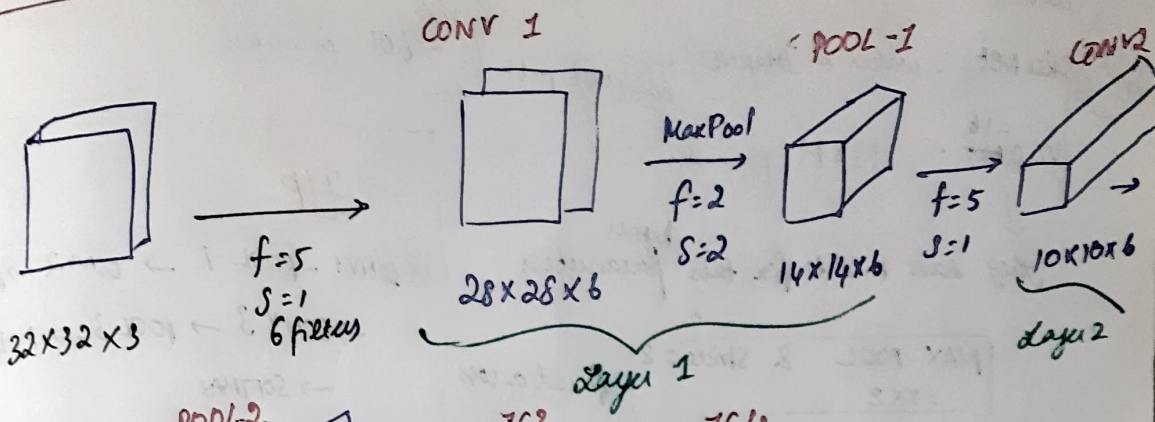
I/P => conv-conv-conv-f

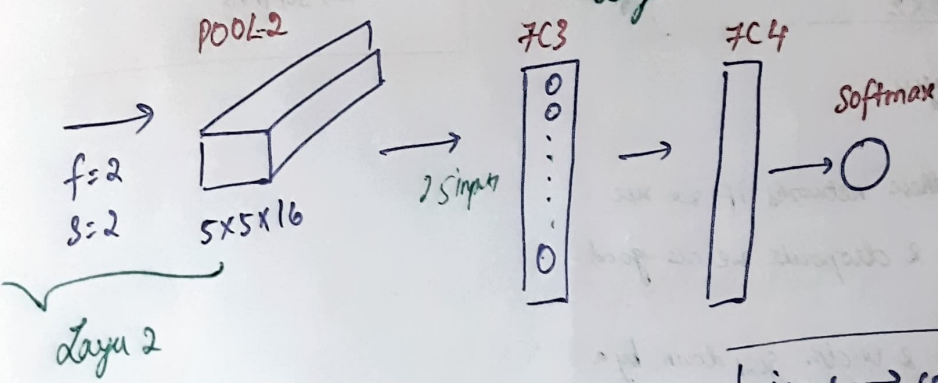
$$\frac{17+2(0)-5}{2}+1 \qquad \frac{12}{2}+1 = 7\times7$$

we try to reduce the dimensions of the image, by increasing the channels

The above example is for 1 sample,
model we could have 1,00,000 samples where half of it
learns discriminatory features b/w 2 class. can be cat & other half are not-cat..

LeNet-5





There is no p, but f & S for pooling

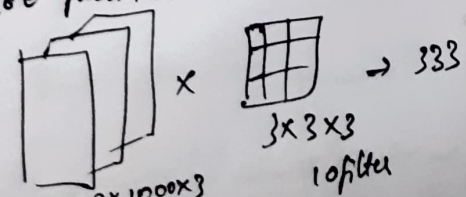
input → conv 1 → pool 1 → conv 2 → pool 2 → FC3 → FC4 → softmax

The no. of neurons in FC3 need not be same as the pixel values in the pool 2.

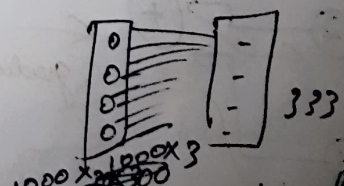
FC3 is a new layer.

20/08/25

280 param in CNN



3M param in ANN



i) Weight sharing - if a filter is expert to pick up a certain feature in one part of the image, the same filter is used to pick up feature on diff part/loc" of the image

ii) Sparsity of connection

$a^{[l+2]}$ = ReLU ($z^{[l+2]}$ + $w a^{[l]}$)

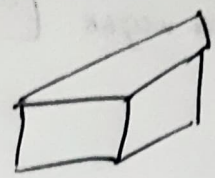
32×1 $\quad$ 32×1 $\quad$ 64×1

$\downarrow$

?

32×64

introduce a matrix 'w' to match the dimensions



$28 \times 28 \times 192$

CONV $5 \times 5 \times 192$

$28 \times 28 \times 32$

#filters = 32

same conv

ResNet - 16 $\rightarrow$ used for transfer learning

Any model to train on images

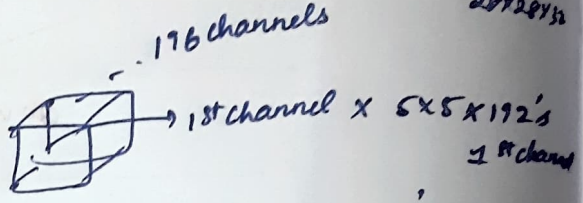
$\downarrow$

try transfer learning using ResNet

or

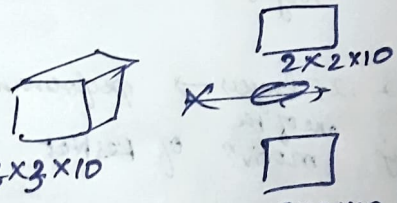
ResNet from scratch.

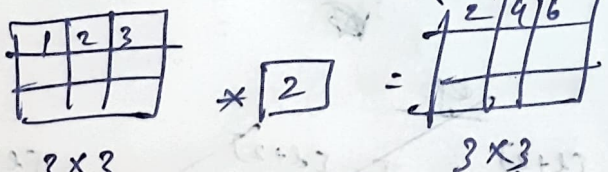
Total ops (multiplication) from $28 \times 28 \times 192$ to $28 \times 28 \times 32$

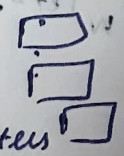


1 X 1 CONV $\rightarrow$ Network in a Network

striding & pooling to reduce h & w of the image.





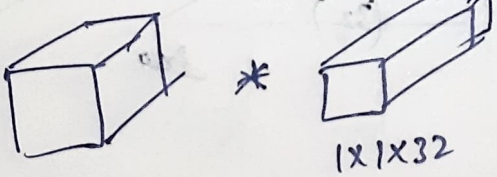


5 filters

1 Channel $\rightarrow$ 10 times

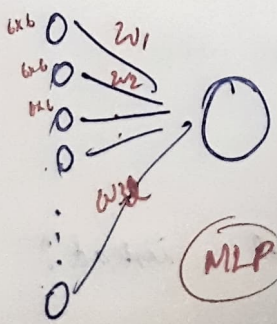
10 channels $\Rightarrow$ $10 \times 10 \times 5$

$= 500$.

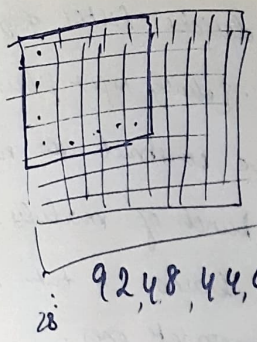


$192 \times 192 \times 32$

$= 1,179,648$

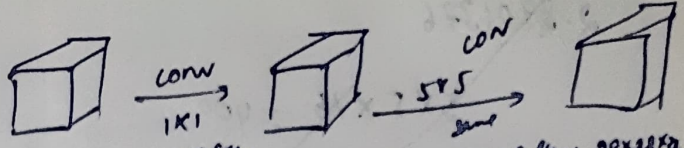


MLP $\rightarrow$ Network in a Network.



28

92,48,44,032



5×5×6×1 ⇒ 900

900 × 192 channels

↓

172800 × 32 filters

⇓

5529600

how many multiplic" are required?

28×28×192 = 150528 (1 filter)

O/p ⇒ 28×28×150528

1 filter = 118013952

16 filters ⇒ 1888223232

Answer:

28×28×192 ⟶ 32 filters of 5×5×192 (already covered by filter)

1 filter 5×5×192 = 4800

O/p stays 28×28

do 1 filter ⇒ 28×28×4800

⇒ 3763200

for 32 filters

↓

32 × 3763200 = 120,422,400 ↘ 120 Million

1×1×16 = 16

28×28×192 = 150528×16

↓

~~1×1×192 for 1 filter~~

⇒

~~28×28×192~~

150528 × 192

⇒ ~~28901376~~

~~for 1st layer~~

= 2408448

⇒ 2.4 million

28/08/25

Helps us in comput" complexity

To get one value in op ⇒ we ~~need~~ have to perfor 5×5×192 . convolution

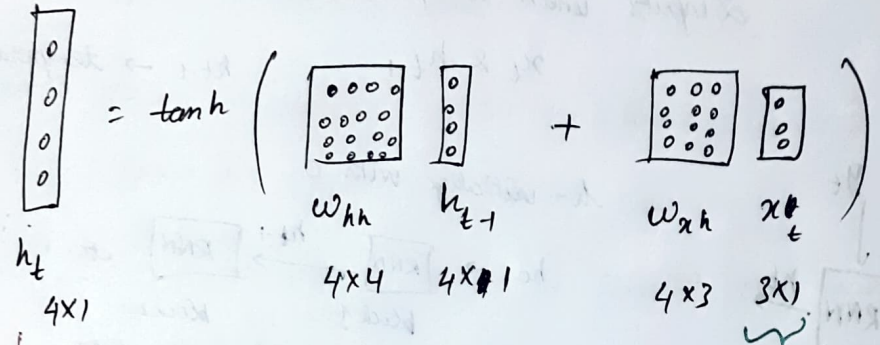
for 28×28×192 pixels we need 5×5×192 × 28×28×32 conv.

⇒ 120M ops (multiplic").

$$h_t = f_W(h_{t-1}, x_t)$$

$$h_t = \tanh\left(W_{hh} h_{t-1} + W_{xh} x_t\right)$$

"No bias term"



↓

We decide the dimension

We fix it same for every hidden state.

↓

This is of fixed size

↓

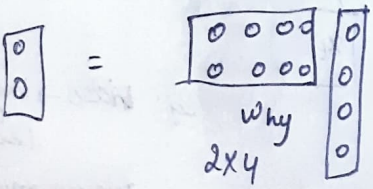
input of 20 words → but we convert it as 3x1.

"Vector rep of every word is same"

"Feature embedding vector size"

$$y_t = W_{hy} h_t$$

Compute h_t & y_t → forward pass ✓



W_{hh} is same across the time steps

17/09/25

Any models (ML) can take vector as an input,

incase of image → do we have spatial info? → so we use CNN → last but 1 layer

↓

feed to RNN (like Transformer)

Character level language module.

If T=3, $X^{(1)} = x_1, x_2, x_3$

T=2 (two time steps)

$X^{(1)}$: $x_1 \in \mathbb{R}^2 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}_{2\times1}$, $x_2 \in \mathbb{R}^2 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}_{2\times1}$

(rep vector is context)

input dim = 2 ⇒ $x_t \in \mathbb{R}^2$

hidden state dim = 3 $\Rightarrow h_t \in \mathbb{R}^3$

$h_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}_{3\times1}$, $W_{xh} = \begin{bmatrix} \cdot & \cdot \\ \cdot & \cdot \\ \cdot & \cdot \end{bmatrix}_{3\times2}$, $W_{hh} = \begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix}$

$y_t \in \mathbb{R}^2$

(o/p at every time point)

$W_{hy} = \begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix}_{2\times3}$

$\begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix}_{W_{hh}} \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}_{h_0} + \begin{bmatrix} \cdot & \cdot \\ \cdot & \cdot \\ \cdot & \cdot \end{bmatrix}_{W_{xh}} \begin{bmatrix} \cdot \\ \cdot \end{bmatrix}_{x_0} = \begin{bmatrix} 2 \\ \cdot \\ \cdot \end{bmatrix}_{3\times1}$

W_{hh}: 3×3, h_0 (h_{t-1}), W_{xh}: 3×2, x_0: 2×1, h_t: output at every timestep

$h_t = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}_{2\times1}$ $h_t = (3\times1)$.

$y_t = \begin{bmatrix} 0 \\ 0 \end{bmatrix}_{2\times1} = \begin{bmatrix} \quad \end{bmatrix}_{W_{hy}\ 2\times3} \begin{bmatrix} \quad \end{bmatrix}_{h_t\ 3\times1}$

$W_{xh} = \begin{bmatrix} 0.5 & -0.3 \\ 0.8 & 0.2 \\ 0.1 & 0.4 \end{bmatrix}_{3\times2}$ → rect

$W_{hh} = \begin{bmatrix} 0.1 & 0.4 & 0.0 \\ -0.2 & 0.3 & 0.2 \\ 0.05 & -0.1 & 0.2 \end{bmatrix}_{3\times3}$ → always sq

$W_{hy} = \begin{bmatrix} 1.0 & -1.0 & 0.5 \\ 0.5 & 0.5 & -0.5 \end{bmatrix}_{2\times3}$ → rect

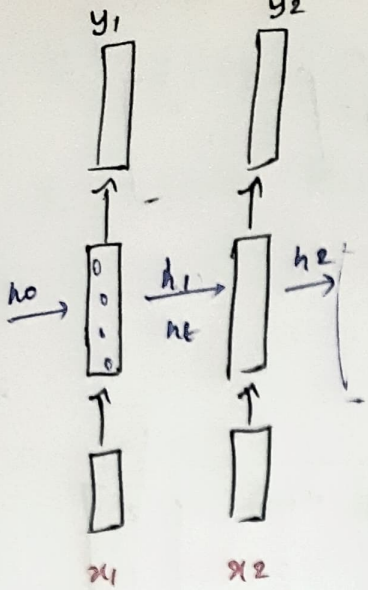
At timestep = 1, t = 1

$h_1 = \tanh(W_{hh} h_{1-1} + W_{xh} x_1)$

$y_1 = W_{hy} h_1$

$\begin{bmatrix} 0.1 & 0.4 & 0 \\ -0.2 & 0.3 & 0.2 \\ 0.05 & -0.1 & 0.2 \end{bmatrix}_{W_{hh}\ 3\times3} \times \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}_{h_0\ 3\times1} = 0$

$\begin{bmatrix} 0+0+0 \\ 0+0+0 \\ 0+0+0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}_{3\times1}$



Compute h_0, ~~h_{t-1}~~ h_2 h_1, y_1

h_2, y_2

At timestep = 2, T=2

$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t)$

$h_2 = \tanh(W_{hh} h_{2-1} + W_{xh} x_2)$

$= \tanh(W_{hh} h_1 + W_{xh} x_2)$

$$\begin{bmatrix} 0.1 & 0.4 & 0 \\ -0.2 & 0.3 & -0.2 \\ 0.05 & -0.1 & 0.2 \end{bmatrix}_{3\times3} \begin{bmatrix} -0.099 \\ 0.833 \\ 0.716 \end{bmatrix}_{3\times1}$$

$$\begin{bmatrix} -0.0099 + 0.3332 + 0 \\ +0.0198 + 0.2499 + 0.1432 \\ -0.00495 + (-0.0833) + 0.1432 \end{bmatrix}$$

$W_{hh} \times h_1$

$$= \begin{bmatrix} 0.3233 \\ 0.4129 \\ 0.22155 \\ 0.05495 \end{bmatrix}_{3\times1}$$

$W_{xh} \times x_1$

$$\begin{bmatrix} 0.5 & -0.3 \\ 0.8 & 0.2 \\ 0.1 & 0.4 \end{bmatrix}_{3\times2} \times \begin{bmatrix} 1 \\ 2 \end{bmatrix}_{2\times1}$$

$$\begin{bmatrix} 0.5 - 0.6 \\ 0.8 + 0.4 \\ 0.1 + 0.8 \end{bmatrix} = \begin{bmatrix} -0.1 \\ 1.2 \\ 0.9 \end{bmatrix}_{3\times1}$$

$$h_1 = \tanh\left(\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} -0.1 \\ 1.2 \\ 0.9 \end{bmatrix} \right)$$

$$h_1 = \tanh\left(\begin{bmatrix} -0.1 \\ 1.2 \\ 0.9 \end{bmatrix} \right)$$

$$h_1 = \begin{bmatrix} -0.099 \\ 0.833 \\ 0.716 \end{bmatrix}_{3\times1}$$

$y_1 = W_{hy} \times h_t$

$y_1 = W_{hy} \times h_1$

$$y_1 = \begin{bmatrix} 1 & -1 & 0.5 \\ 0.5 & 0.5 & -0.5 \end{bmatrix}_{2\times3} \begin{bmatrix} -0.099 \\ 0.833 \\ 0.716 \end{bmatrix}_{3\times1}$$

$$= \begin{bmatrix} 0.099 - 0.833 + 0.358 \\ -0.0495 + 0.4165 - 0.358 \end{bmatrix}$$

$$y_1 = \begin{bmatrix} -0.574 \\ 0.009 \end{bmatrix}_{2\times1}$$

$$h_2 = \tanh\left(\begin{bmatrix} 0.3233 \\ 0.4733 \\ 0.22155 \end{bmatrix} \right)$$

$$= \begin{bmatrix} 0.312 \\ 0.456 \end{bmatrix}$$

$W_{xh} \times x_2 = \begin{bmatrix} 0.5 & -0.3 \\ 0.8 & 0.2 \\ 0.1 & 0.4 \end{bmatrix}_{3\times2} \begin{bmatrix} -1 \\ 1 \end{bmatrix}_{2\times1} = \begin{bmatrix} -0.5 - 0.3 \\ -0.8 + 0.2 \\ -0.1 + 0.4 \end{bmatrix}_{3\times1}$

$h_2 = \tanh\left(\begin{bmatrix} 0.3233 + (-0.8) \\ -0.4127 - 0.6 \\ 0.05495 + 0.3 \end{bmatrix} \right) = \begin{bmatrix} -0.8 \\ -0.6 \\ 0.3 \end{bmatrix}$

$h_2 = \tanh\left(\begin{bmatrix} 0.4767 \\ -0.2267 \\ 0.52155 \end{bmatrix} \right) = \begin{bmatrix} -0.4435 \\ -0.2228 \\ 0.4788 \end{bmatrix}$

Non-linear activ^n ↓ tanh because ReLU wasn't der.

$y_2 = h_2 = \tanh\left(\begin{bmatrix} -0.4435 \\ -0.1871 \end{bmatrix} \right)$ $\quad h_2 = \tanh\left(\begin{bmatrix} -0.4767 \\ -0.1871 \\ 0.35495 \end{bmatrix} \right)$

$h_2 = \tanh\left(\begin{bmatrix} -0.4435 \\ -0.1849 \\ 0.3407 \end{bmatrix} \right)$

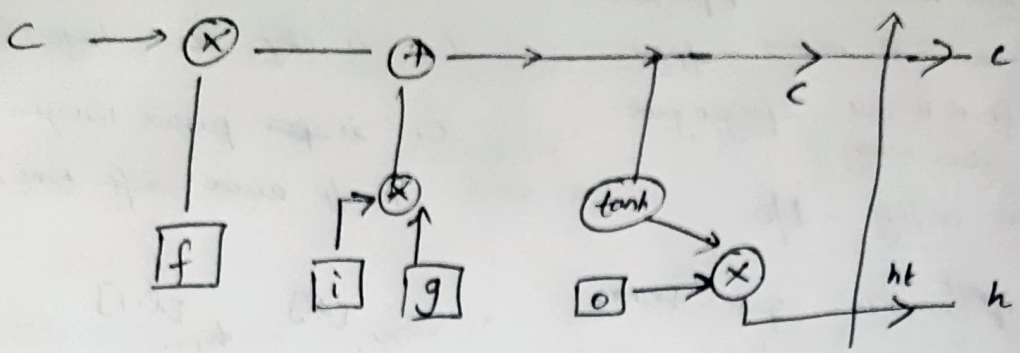
$y_2 = W_{hy} \times h_2$

$= \begin{bmatrix} 1 & -1 & 0.5 \\ 0.5 & 0.5 & -0.5 \end{bmatrix}_{2\times3} \begin{bmatrix} -0.4435 \\ -0.1849 \\ 0.3402 \end{bmatrix}_{3\times1}$

$= \begin{bmatrix} -0.4435 + 0.1849 + 0.17035 \\ -0.2217 - 0.09245 - 0.1703 \end{bmatrix} = \begin{bmatrix} -0.08825 \\ -0.4845 \end{bmatrix}$

Forward pass ends.

W_{hh} & W_{xh} is same → parameters are shared across time steps.



24/09/25 ✱ LSTM minimizes & not 100%. solve vanishy gradian.

Example

$x_t = [0.5, -0.1]^T$ ✓ $\mathbb{R}^2$

$h_{t-1} = [0.0, 0.1]^T$ $\mathbb{R}^2$

$C_{t-1} = [0.2, -0.2]^T$ $\mathbb{R}^2$

$W_{xi} = \begin{bmatrix} 0.5 & -0.3 \\ 0.4 & 0.1 \end{bmatrix}$

$W_{hi} = \begin{bmatrix} 0.1 & 0.2 \\ -0.2 & 0.05 \end{bmatrix}$

$W_{xf} = \begin{bmatrix} -0.4 & 0.2 \\ 0.3 & 0.3 \end{bmatrix}$

$W_{hg} = \begin{bmatrix} 0.2 & 0.1 \\ -0.1 & 0.05 \end{bmatrix}$

$W_{hf} = \begin{bmatrix} 0.05 & -0.1 \\ 0.2 & 0.1 \end{bmatrix}$

$h_t = ?$

$C_t = ?$

$W_{xo} = \begin{bmatrix} 0.3 & 0.25 \\ -0.2 & 0.2 \end{bmatrix}$

$W_{xg} = \begin{bmatrix} -0.5 & 0.4 \\ 0.2 & -0.3 \end{bmatrix}$

$W_{ho} = \begin{bmatrix} 0.15 & 0.05 \\ 0.1 & -0.2 \end{bmatrix}$

Formulae: → Haddamad transfor

$C_t = f_t \odot C_{t-1} + i_t \odot g_t$

$h_t = o_t \odot \tanh(C_t)$

$f_t = \sigma(W_{hf} h_{t-1} + W_{xf} x_t)$

$i_t = \sigma(W_{hi} h_{t-1} + W_{xi} x_t)$

$o_t = \sigma(W_{ho} h_{t-1} + W_{xo} x_t)$

$g_t = \tanh(W_{hg} h_{t-1} + W_{xg} x_t)$

$C_t = f_t \odot C_{t-1} + i_t \odot g_t$

$f_t \Rightarrow \sigma\left(\begin{bmatrix} 0.05 & -0.1 \\ 0.2 & 0.1 \end{bmatrix} \cdot \begin{bmatrix} 0.0 \\ 0.1 \end{bmatrix}\right)$ (2×2)

$f_t = \sigma\left(\begin{bmatrix} 0.05 & -0.1 \\ 0.2 & 0.1 \end{bmatrix} \begin{bmatrix} 0.0 \\ 0.1 \end{bmatrix}\right)$

W_{hf} 2×2 h_{t-1} 2×1

$\begin{bmatrix} 0 + -0.01 \\ 0 + 0.01 \end{bmatrix} = \begin{bmatrix} -0.01 \\ 0.01 \end{bmatrix}_{2\times1}$

$W_{hf} \times x_t$ $\begin{bmatrix} -0.4 & 0.2 \\ 0.3 & 0.3 \end{bmatrix}$ $\begin{bmatrix} 0.5 \\ -0.1 \end{bmatrix}$

2×2 2×1

$\Rightarrow \begin{bmatrix} 0.2 + (-0.02) \\ 0.15 - 0.03 \end{bmatrix}$

$$\Rightarrow \begin{bmatrix} 0.18 \\ 0.12 \end{bmatrix}_{2\times1}$$

$$\begin{bmatrix} -0.01 \\ 0.01 \end{bmatrix} + \begin{bmatrix} 0.18 \\ 0.12 \end{bmatrix} = \begin{bmatrix} 0.17 \\ 0.13 \end{bmatrix}_{2\times1}$$

$$f_t = \sigma\begin{pmatrix} 0.17 \\ 0.13 \end{pmatrix}$$

$$\frac{1}{1+e^{-0.17}} \Rightarrow \frac{1}{1+0.843} = \frac{1}{1.843} = 0.542$$

$$f_t = \begin{pmatrix} 0.542 \\ 0.532 \end{pmatrix}_{2\times1}$$

$$i_t = w_{hi} \times h_{t-1}$$

$$\begin{bmatrix} 0.1 & 0.2 \\ -0.2 & 0.05 \end{bmatrix} \times \begin{bmatrix} 0.0 \\ -0.1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 + 0.02 \\ 0 + 0.005 \end{bmatrix} = \begin{bmatrix} 0.02 \\ 0.005 \end{bmatrix}$$

$$w_{xi}\, x_t$$

$$\begin{bmatrix} 0.5 & -0.3 \\ 0.4 & 0.1 \end{bmatrix} \begin{bmatrix} 0.5 \\ -0.1 \end{bmatrix}$$

$$\begin{bmatrix} 0.25 + 0.03 \\ 0.20 + 0.01 \end{bmatrix} = \begin{bmatrix} 0.28 \\ 0.21 \end{bmatrix}$$

$$i_t = \sigma\begin{pmatrix} 0.3 \\ 0.215 \end{pmatrix}$$

$$i_t = \sigma\begin{pmatrix} 0.574 \\ 0.553 \end{pmatrix}$$

$$\frac{1}{1.740}$$

$$\frac{1}{1.806}$$

$$g_t \Rightarrow w_{hg} \times h_{t-1}$$

$$\begin{bmatrix} 0.2 & 0.1 \\ -0.1 & 0.05 \end{bmatrix} \times \begin{bmatrix} 0.0 \\ 0.1 \end{bmatrix}$$

$$\begin{bmatrix} 0 + 0.01 \\ 0 + 0.005 \end{bmatrix} = \begin{bmatrix} 0.01 \\ 0.005 \end{bmatrix}$$

$$\begin{bmatrix} -0.5 & 0.4 \\ 0.2 & -0.3 \end{bmatrix} \times \begin{bmatrix} 0.5 \\ -0.1 \end{bmatrix}$$

$$w_{xg} \times x_t$$

$$\Rightarrow \begin{bmatrix} -0.25 + (-0.04) \\ 0.1 + 0.03 \end{bmatrix}$$

$$= \begin{bmatrix} 0.29 \\ 0.13 \end{bmatrix}$$

$$\begin{bmatrix} 0.29 + 0.0 \\ 0.13 + 0.005 \end{bmatrix}$$

$$= \begin{bmatrix} -0.27 \\ 0.134 \end{bmatrix}$$

$$\tanh \rightarrow$$

$$g_t \Rightarrow \begin{bmatrix} -0.27 \\ 0.134 \end{bmatrix}$$

$$c_t = \begin{pmatrix} 0.542 \\ 0.532 \end{pmatrix} \odot \begin{pmatrix} 0.2 \\ -0.2 \end{pmatrix}$$

f_t $\qquad$ c_{t-1}

$$c_t = \begin{bmatrix} 0.088 \\ -0.1064 \end{bmatrix}$$

$$o_t = w_{ho} \times h_{t-1} \begin{bmatrix} 0.15 & 0.05 \\ 0.1 & -0.2 \end{bmatrix} \begin{bmatrix} 0.0 \\ 0.1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 + 0.005 \\ 0 + (-0.02) \end{bmatrix} = \begin{bmatrix} 0.005 \\ -0.02 \end{bmatrix}$$

$W_{xo}\, x_t = \begin{bmatrix} 0.3 & 0.25 \\ -0.2 & 0.2 \end{bmatrix} \begin{bmatrix} 0.5 \\ -0.1 \end{bmatrix}$

$= 0.15 + (-0.025) = \begin{bmatrix} -0.175 \\ -0.11 \end{bmatrix}$

$-0.1 + (-0.01)$

$o_t = \sigma \begin{bmatrix} 0.005 - 0.175 \\ -0.02 - 0.11 \end{bmatrix} = \sigma \begin{bmatrix} 0.17 \\ 0.13 \end{bmatrix} = \begin{bmatrix} 0.542 \\ 0.532 \end{bmatrix}$

But correct o_t is $\begin{bmatrix} 0.5 \\ 0.4 \end{bmatrix}$

$h_t = o_t \odot \tanh(c_t)$

$c_t \Rightarrow \begin{bmatrix} -0.06 \\ -0.03 \end{bmatrix}$ $\tanh \begin{bmatrix} -0.06 \\ -0.03 \end{bmatrix} \Rightarrow \begin{bmatrix} -0.059 \\ -0.029 \end{bmatrix}$

$h_t \begin{bmatrix} 0.53 \\ 0.46 \end{bmatrix} \odot \begin{bmatrix} -0.059 \\ -0.029 \end{bmatrix} = \begin{bmatrix} -0.03127 \\ -0.01334 \end{bmatrix}$

$h_t = \begin{bmatrix} -0.03127 \\ -0.01334 \end{bmatrix}$

Backprop:

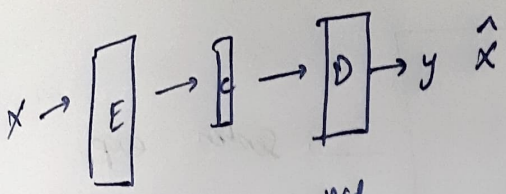
$c_t \overset{\text{LSTM}}{=} f_t \odot c_{t-1} + i_t \odot g_t$

$i_t \odot g_t$ — similar to RNN exponential decay even if this goes to 0, the term will decay.

RNN

$\frac{\partial h}{\partial h_{t-1}} = \tanh'(W_{hh}\, h_{t-1} + W_{xh}\, x_t)$

30/09/25 Lecture 40



We Wd

learn the weights We, Wd

I need to tweak We & Wd in such a way that we are getting the input (ideally) $\hat{x}$

$L = \frac{1}{2}\|x - \hat{x}\|_2$

then, how is it going to learn

↓ backprop

$\frac{\partial L}{\partial W_e}, \frac{\partial L}{\partial W_d}$

ideally we need smaller dim context vector.

Q) $x \in \mathbb{R}^2$, $x = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$

$h \in \mathbb{R}^1$

↓ context vector.

What shd be the size of We & Wd

$\begin{bmatrix} 2 \\ 0 \end{bmatrix}_{2\times1}$ $[X]_{1\times1}$

x h

prove the linear act" function is equal to PCA. (X)

by replacing act" by linear act".

$W_e = h$ ~~We = 1X2~~

Under 3 cond", PCA = Autoencoder

i) when encoder & decoder are linear block

ii) MSE is the loss func".

$W_d = 2\times1 = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix}_{2\times1}$

Calc fwd pass

Encoder

$W_e \Rightarrow \begin{bmatrix} 0.5 & -1.0 \end{bmatrix}_{1\times2} \begin{bmatrix} 2 \\ 0 \end{bmatrix}_{2\times1}$

$= [1+0] = [1]$

$h = [1]_{1\times1}$ 2D → 1D

$h \times W_d = \hat{x}$

$1\times1 \quad 2\times1$

$W_d \times h = \hat{x}$

$2\times1 \quad 1\times1$

$\begin{bmatrix} 1 \\ 0.5 \end{bmatrix}_{2\times1} \times [1]_{1\times1} = 1 + 0.5 = 1.5$

$\hat{x} = \begin{bmatrix} 1 \\ 0.5 \end{bmatrix}$

$\frac{1}{2}(2-1)^2 = \frac{1}{2}\times1 = 0.5$

$\frac{1}{2}(0-0.5)^2$

$\Rightarrow 0.125$

$\therefore L = \begin{bmatrix} 0.5 \\ 0.125 \end{bmatrix}$

$W_e = \begin{bmatrix} 0.5 & -1.0 \end{bmatrix}_{1\times2}$ Total loss = 0.625

Ans: 0.625

$\alpha_{t,j}$ ⇒ diff (dot product) score fn"
to better capture similarity (new paper).

Numerical

$h_1 = [1, 0, 1]$

$h_2 = [0, 1, 1]$

$h_3 = [1, 1, 0]$

$S_{t-1} = [1, 0, 1]$

Score fn = dot product

$c_t = ?$

$$c_t = \sum_{j=1}^{T} \alpha_{t,j} h_j$$

$$c_t = \sum_{j=1}^{3} \alpha_{t,1} h_1 + \alpha_{t,2} h_2 + \alpha_{t,3} h_3$$

$$= \alpha_{t,1} h_1 + \alpha_{t,2} h_2 + \alpha_{t,3} h_3$$

Assume $t=3$

$$\alpha_{t,1} = \frac{\exp(\text{score}(S_{t-1}, h_1))}{\exp(\text{score}(S_{t-1}, h_1)) + \exp(\text{score}(S_{t-1}, h_2)) + \exp(\text{score}(S_{t-1}, h_3))}$$

$$= \frac{\text{Numerator}}{[1,0,1] \cdot [1,0,1]}$$

S_{t-1} $\quad$ h_1

$\Rightarrow \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}_{3\times 1} [1, 0, 1]_{1\times 3}$

$= 1 + 0 + 1 = 2$

$[1, 0, 1]_{1\times 3} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}_{3\times 1}$

$\Rightarrow 1 + 0 + 1 \Rightarrow [2]_{1\times 1}$

$$\frac{\exp(2)}{\exp(2) + \exp(1) + \exp(1)}$$

$\Rightarrow [1, 0, 1]_{S_{t-1}} \cdot h_2 \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$

$0 + 0 + 1 \Rightarrow [1]_{1\times 1}$

$$\Rightarrow \frac{7.389}{7.389 + 2.718 + 2.718}$$

$$= \frac{7.389}{12.825} = \boxed{0.576}$$

$\Rightarrow [1, 0, 1]_{S_{t-1}} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}_{h_3}$

$= 1 + 0 + 0$

$\alpha_{t,1} = 0.576$

$\alpha_{t,2} \Rightarrow \frac{2.718}{7.389 + 2.718 + 2.718}$

$= 0.211$

$\alpha_{1,3} = \frac{2.718}{7.389 + 2.718 + 2.718}$

$= 0.211$

$C_t = \alpha_{t,1} h_1 + \alpha_{t,2} h_2 + \alpha_{t,3} h_3$

$= 0.576 \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} + 0.211 \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} + 0.211 \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$

$\Rightarrow \begin{bmatrix} 0.576 \\ 0 \\ 0.576 \end{bmatrix} + \begin{bmatrix} 0 \\ 0.211 \\ 0.211 \end{bmatrix} + \begin{bmatrix} 0.211 \\ 0.211 \\ 0 \end{bmatrix}$

$C_t \Rightarrow \begin{bmatrix} 0.787 \\ 0.422 \\ 0.787 \end{bmatrix}$

S = vector
h = vector
C = vector.
t

RDBMS (comput. complex + tough to query)
↓
Vector DB

NoSQL db
↓
MongoDB
Cassandra

key - vector | value - any string

Pinecone, FAISS.

Before vector db → Google → principle of inverted indices

key	value
action	url1, url2, url4, url N
movie	url2, url80, url4 ...

intersection

page ranking - linear algebra

after retrieval → ranking (Stanford PHD)

using eigenvectors & eigenvalues.

Vector db + RAGs ...

db retrieval problem ↖ Attention mechanism can be seen as in Vector databases.

Attention is nothing but a similarity computation

Example ⇒

Oct/7/25

Search query

query, indexing to table, retrieve the corresponding results.

Google search → vector db.

dot pdt of query vector with every key vector. ⇒ then rank it ..

Key	Value
[1.0, 0.5, 0.2]	"Action movie Die Hard"
[0.8, 1.0, 0.1]	"Romance Titanic"
[0.9, 0.4, 0.8]	"Action movie: John Wick"

Query = looking for action movies

⇓ Word2Vec

[1.0, 0.3, 0.5]

$$\begin{bmatrix}1\\0.5\\0.2\end{bmatrix}_{3\times1} \quad [1\ 0.3\ 0.5]_{1\times3} \rightarrow 1$$

$$[1\ 0.5\ 0.2]_{1\times3}\begin{bmatrix}1\\0.3\\0.5\end{bmatrix}_{3\times1} = 1+0.15+0.10 = 1.25 \quad (2)$$

$$[0.3\ 1\ 0.1]\begin{bmatrix}1\\0.3\\0.5\end{bmatrix} = 0.3+0.3+0.05 = 0.65 \quad (3)$$

$$[0.9\ 0.4\ 0.8]\begin{bmatrix}1\\0.3\\0.5\end{bmatrix} = 0.9+0.12+0.40 = 1.42 \quad (1)$$

Transformer

It's also a encoder decoder network.

cross-attention ⇒ from the s_{t-1} to all inputs.

self-attention → multihead attention.

RNN maintained time steps → has recurrence

transformers doesn't have recurrence by itself → which is a disadvantage

⇓

positional encoding

Input + Positional encoding → Encoder block → Multihead attention

2 sub components → FFN

+ Skip connection + Add & Norm + Z-score

3 sub components in decoder block → Masked multihead attention → Multihead attention → FFN

09/10/25

THEORY

Matrix mul

Embedding W^Q = Query Vector

using embedding vec → Query
→ Key
→ Val

"self attention" → key & value are exactly identical

Treating

Thinking | Machines

① q_1 = Thinking | ① q_1 = Machines
k_1 = Thinking | k_1 = Thinking
v_1 = Thinking | v_1 = "
② q_1 = Thinking | ② q_1 = Machines
k_2 = Machine | k_2 = Machines
v_2 = " | v_2 = "

But technically k_1 & v_1 , k_2 & v_2 ... will be diff coz it is multiplied by diff weight matrices W_{q_1}, W_{v_1}, W_{k_1} etc.

Example

i/p: Playing outside → z_1, z_2

Playing $q_1 = [0.212 \;\; 0.04 \;\; 0.63 \;\; 0.36]^T$

$k_1 = [0.31, 0.84, 0.963, 0.57]^T$

$v_1 = [0.36, 0.83, 0.1, 0.38]^T$

Outside $q_2 = [0.1, 0.14 \;\; 0.86 \;\; 0.77]^T$

$k_2 = [0.45 \;\; 0.94 \;\; 0.73 \;\; 0.58]^T$

$v_2 = [0.31, .36, 0.19, 0.72]^T$

$q_1 \times k_1 =$ $\begin{bmatrix} 0.212 \\ 0.04 \\ 0.63 \\ 0.36 \end{bmatrix}$ $[0.31 \;\; 0.84 \;\; 0.963 \;\; 0.57]$ (4×1 × 1×4)

$\begin{bmatrix} 0.212 & 0.04 & 0.63 & 0.36 \end{bmatrix}$ (1×4) $\begin{bmatrix} 0.31 \\ 0.84 \\ 0.963 \\ 0.57 \end{bmatrix}$ (4×1)

$\Rightarrow 0.06572 + 0.0336 + 0.60669 + 0.2052$

$q_1 \times k_1 = 0.91121$

$$q_1 \times k_2 = [0.212 \quad 0.04 \quad 0.63 \quad 0.36] \begin{bmatrix} 0.45 \\ 0.94 \\ 0.73 \\ 0.58 \end{bmatrix}$$

$$= 0.0954 + 0.0376 + 0.4599 + 0.2088 = 0.8017$$

$d_k = 4$ divide by 2

$\sqrt{d_k} = 2$

$$q_1 \times k_1 = \frac{0.91121}{2} = 0.455605 \Rightarrow \frac{e^{0.455}}{e^{0.455} + e^{0.423}} = \frac{1.574}{1.614} = 0.975$$

$$q_1 \times k_2 = \frac{0.5467}{2} = 0.42335 \Rightarrow \frac{1}{1.4485} = 0.6903$$

Softmax x val

$0.61199 \times V_1 \Rightarrow [0.36 \quad 0.83 \quad 0.1 \quad 0.38]$

0.2203

Softmax

$$\frac{e^{0.455}}{e^{0.455} + e^{0.423}} = \frac{1.5761}{1.5761 + 1.5265} = \frac{1.5761}{3.102} = 0.5080$$

Softmax x value $\frac{e^{0.423}}{e^{0.455} + e^{0.423}} = \frac{1.5265}{3.102} = 0.4921$

$S \times V_1$ $0.5080 [0.36 \quad 0.83 \quad 0.1 \quad 0.38] \Rightarrow [$

$S \times V_2$

$$S_1 \times V_1 = 0.5080 \begin{bmatrix} 0.36 \\ 0.83 \\ 0.1 \\ 0.38 \end{bmatrix} = \begin{bmatrix} 0.182 \\ 0.421 \\ 0.0508 \\ 0.193 \end{bmatrix}$$

$$S_2 \times V_2 = 0.4921 \begin{bmatrix} 0.31 \\ 0.36 \\ 0.19 \\ 0.72 \end{bmatrix} = \begin{bmatrix} 0.1525 \\ 0.1771 \\ 0.0934 \\ 0.3543 \end{bmatrix}$$

$S_1 V_1 + S_2 V_2 \Rightarrow \begin{bmatrix} 0.4334 \\ 0.5981 \\ 0.2534 \\ 0.5473 \end{bmatrix} \Rightarrow 0.1442 \Rightarrow Z_1$

For Outside

$q_2 \times k_1 = \begin{bmatrix} 0.1 & 0.14 & 0.86 & 0.77 \end{bmatrix} \times \begin{bmatrix} 0.31 \\ 0.84 \\ 0.963 \\ 0.57 \end{bmatrix}$

$= 0.031 + 0.1176 + 0.82818 + 0.4389 \Rightarrow 1.41568$

$q_2 \times k_2 = \begin{bmatrix} 0.1 & 0.14 & 0.86 & 0.77 \end{bmatrix} \times \begin{bmatrix} 0.45 \\ 0.94 \\ 0.73 \\ 0.58 \end{bmatrix}$

$= [0.045 + 0.1316 + 0.6278 + 0.4466] = 1.6526$

$s_1 = \frac{q_1 \times k_1}{2} = \frac{1.41568}{2} = 0.70784$ → Softmax $\frac{2.029}{2.029 + 2.284} = 0.4704$

$s_2 = \frac{q_2 \times k_2}{2} = \frac{1.6526}{2} = 0.8263$ $\frac{2.284}{2.029 + 2.284} = 0.529$

$s_1 \times v_1 = 0.4704 \times \begin{bmatrix} 0.36 \\ 0.83 \\ 0.21 \\ 0.1 \\ 0.0508 \\ 0.38 \\ 0.193 \end{bmatrix} = \begin{bmatrix} 0.085 \\ 0.1980 \\ 0.0238 \\ 0.0907 \end{bmatrix}$

$s_2 \times v_2 = 0.529 \times \begin{bmatrix} 0.31 \\ 0.36 \\ 0.19 \\ 0.72 \end{bmatrix} = \begin{bmatrix} 0.1639 \\ 0.1904 \\ 0.10057 \\ 0.3808 \end{bmatrix}$

$s_1 v_1 + s_2 v_2 = \begin{bmatrix} 0.3332 \\ 0.5808 \\ 0.1475 \\ 0.5595 \end{bmatrix}$

$\begin{bmatrix} 0.1693 \\ 0.3704 \\ 0.04704 \\ 0.1787 \end{bmatrix}$

Ans: $Z_2 = [0.30\ 0.57\ 0.14\ 0.54]$

Multihead attention

Wherever we need to aggregate multiple vectors ⇒ use fc layer.

Input machine thinking → X → Embed each word → Split to 8 heads → Calculate Z

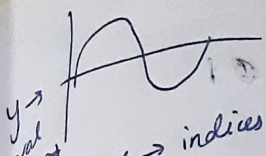
8 → W_0^Q, W_0^K, W_0^V, Q_0 → Z_0

R → W_1^Q, W_1^K, W_1^V → Z_1, Z_2, ..., Z_8

→ Concatenate the Z matrices then multiply with $\begin{bmatrix} W_0 \\ \\ \end{bmatrix}$ → (Z) final rep vector.

Classifier for transformer → after fc layer → SVMs or ...

many to many means → decoder network



$x \rightarrow$ indices

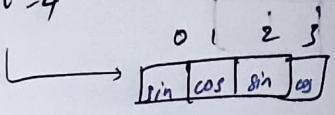
$y \rightarrow$ sin value of pos^n

It maintains the notion of relative ordering.

Key comp of Transf. Arch

0 1 2 3 4

$t = 4$



Pos^n embedding vector

$P_t \in \mathbb{R}^{512}$

$0 < i < d/2$

for pos^n 3

$PE_{3,2i}$ for $i = 0$ we can fill two pos^n simultaneously

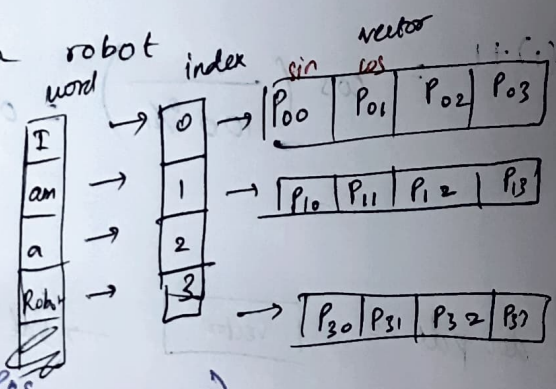
$2(0) := 0 \rightarrow$ sin

$2(0)+1 \Rightarrow 1 \rightarrow$ cos & so on

PE is a one-time job $\rightarrow$ need not parallelize.

---

eg: I am a robot



P_E : $d = 4$ or $\mathbb{R}^4$

dim of pos emb = 4

$$P_{pos,2i} = \sin\left(\frac{pos}{10000^{(2i/\mathbb{R}d)}}\right)$$

$\Downarrow$ eg: do this 100 over

$$P_{pos,2i+1} = \cos\left(\frac{pos}{10000^{(2i/dem b)}}\right)$$

~~for i = 0~~ for pos = 0

$$P_{0,2(0)} = \sin\left(\frac{0}{100^{\left(\frac{2\times0}{4}\right)}}\right) = \sin\left(\frac{0}{100^{0}}\right) = \sin(0) = 0$$

$P_{00} = 0$

$P_{0,2(0)+1} = \cos\left(\frac{0}{100^{0/4}}\right) \rightarrow \cos(0) = 1$

$P_{01} = 1$

for $i=1$

$P_{0,2(1)} = \sin\left(\frac{0}{100^{2(1)/4}}\right) = \sin\left(\frac{0}{100^{1/2}}\right) = 0.$

$P_{02} = 0$

$P_{0,2(1)+1} = \cos\left(\frac{0}{100^{2/4}}\right) = 1 \quad P_{03} = 1$

$\Rightarrow$ | 0 | 1 | 0 | 1 |

For Pos = 2 index 1.

$P_{1,2(0)} \overset{i=0}{=} \sin\left(\frac{1}{100^{2(0)/4}}\right) \rightarrow \sin\left(\frac{1}{10}\right) = 0.017$

$P_{1,2(0)+1} = \cos\left(\frac{1}{100^{0/4}}\right) = \cos(1) = 0.999$

For $i=1$

$P_{1,2(1)} = \sin\left(\frac{1}{100^{2/4}}\right) = \sin\left(\frac{1}{10}\right) = 0.0017$

$P_{1,2(1)+1} = \cos\left(\frac{1}{100^{2/4}}\right) = 0.999$

$\vdots$

P_{500} } dot pdt = [vector] $\rightarrow$ val

P_{501}

P_{10000} } dot pdt = [vector]

now we got

input = [ ]

pos = [ ] } concatenate

or add [ ] + [ ] =

Translomes paper has this

$X \Rightarrow$ random variable takes values 0, 1, 2.

Gn P distb^{n} ; takes 0 = Prob = 0.36
1 = Prob = 0.98
2 = " = 0.16

Distrib X	0	1	2
P(X)	9/25	12/25	4/25
Q(X)	1/3	1/3	1/3

Ans: $D_{KL}(P||Q) = \sum_{x \in X} P(X=x) \ln \frac{P(X=x)}{Q(X=x)}$

$D_{KL}(P||Q) = \frac{9}{25} \ln\left(\frac{9/25}{1/3}\right) + \frac{12}{25} \ln\left(\frac{12/25}{1/3}\right) + \frac{4}{25} \ln\left(\frac{4/25}{1/3}\right)$

$D_{KL}(P||Q) = 0.0852996$

NOTE:
If D_{KL} equal to 0, then P = Q (which we don't want)
more closer D_{KL} to zero, more similar P & Q are.

$D_{KL}(Q||P) = \frac{1}{3} \ln\left(\frac{1/3}{9/25}\right) + \frac{1}{3} \ln\left(\frac{1/3}{12/25}\right) + \frac{1}{3} \ln\left(\frac{1/3}{4/25}\right)$

$D_{KL}(Q||P) = 0.09745$

$D_{KL}(P||Q) \neq D_{KL}(Q||P)$

⇒ KL divergence is not symmetric (always). Since D_{KL} is close to zero, that's why P & Q can be considered similar.

(conclusion from this qn ??)

Generative models

Fully visible models
(generate obs data directly from i/p)

Latent variable models
(define hidden variables to generate observed data)